

Vorlesung

Mensch-Maschine-Interaktion

WS 2024/2025



Kapitel F.

Geometrisches Modellieren

Priv.-Doz. Dr. Frank Weichert
Lehrstuhl für Computergraphik (Informatik VII)
Technische Universität Dortmund
<https://graphics.cs.tu-dortmund.de>

Hinweis:

„Der Nutzerin/dem Nutzer ist bekannt, dass die nachfolgenden Inhalte und Materialien urheberrechtlich geschützt sind. Die Nutzung ist ausschließlich für den persönlichen Gebrauch im Rahmen von universitären Zwecken zulässig. Insbesondere ist die Vervielfältigung, Bearbeitung, Verbreitung und jede Art der Verwertung sowie die Weitergabe an Dritte nicht gestattet. Zuwiderhandlungen werden zivil- und strafrechtlich verfolgt.“

F.1. Einleitung

- F.1.1. Motivation
- F.1.2. Grundkonzepte

F.2. Polynombasierte Repräsentationen

- F.2.1. Parameterdarstellung einer Kurve
- F.2.2. Implizite Darstellung einer Kurve
- F.2.3. Parameterdarstellung einer Fläche
- F.2.4. Explizite Darstellung von Flächen
- F.2.5. Implizite Darstellung von Flächen
- F.2.6. Polynomkurvensegmente
- F.2.7. Bézier-Kurvensegmente
- F.2.8. Bézier-Flächensegmente

F.3. Spline-basierte Repräsentation

- F.3.1. B-Spline-Funktionen
- F.3.2. Eigenschaften
- F.3.3. Knotenvektor
- F.3.4. Kontrollierbarkeit

F.4. Polygonbasierte Repräsentationen (Netze)

- F.4.1. Zellkomplexe (Netze)
- F.4.2. Datenstrukturen für Netze
- F.4.3. Polygonale Approximation von Kurven und Flächen
- F.4.4. Triangulierung
- F.4.5. Voronoi-Diagramm
- F.4.6. Delaunay-Triangulierung
- F.4.7. Netzverbesserung
- F.4.8. Netzreduktion

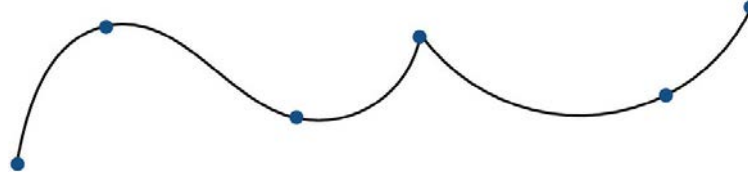
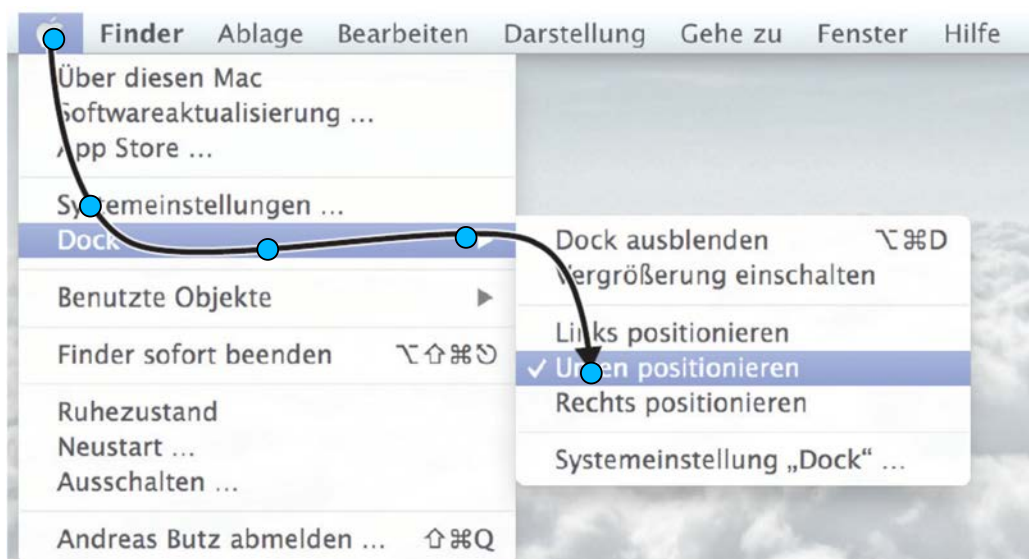
Motivation:

1. Kontrollierung von realen Bewegungen durch wenige Kontrollpunkte
2. Effiziente Repräsentation und Manipulation von Objekte

Lineare Interpolation gewährt keinen natürlichen Verlauf

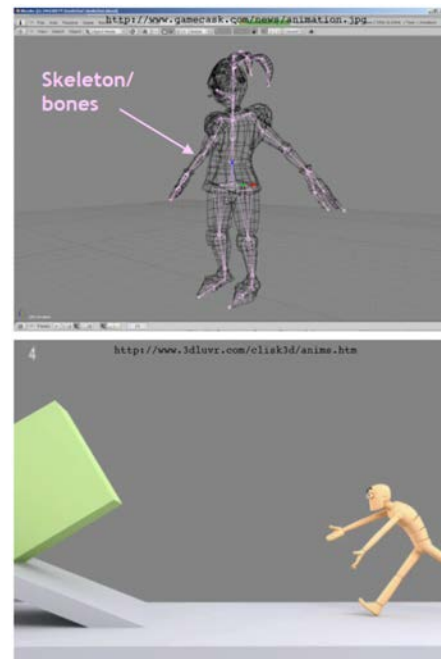


„Splines“ für natürliche (weiche) und kontrollierbare Interpolation

**Repräsentation von Interaktionspfaden**

Skelett-Animation

- Definieren eines Skeletts für ein Polygonnetz
→ Topologie/Struktur des Modells
- Bewegung nur durch die Knochen dieses Skeletts
 - durch Keyframing von Gelenkwinkeln
 - durch Motion-Capture-Daten
 - durch inverse Kinematik
- Polygonnetz folgt und verformt sich
 - Verbindung zwischen Knochen und Netz ist nicht starr
 - Mesh bleibt geschlossen und glatt



Echtzeit-Rendering und Bewegungserfassung

- Positionsverfolgung und/oder Ausrichtung der
 - Gliedmaßen eines Akteurs
 - Merkmalspunkte eines Gesichts
 - optischen Markierungen auf einem Anzug
- Definieren einer Beziehung zwischen getrackten Merkmalspunkten und 3D-Szenenpunkten
- „Virtuelle Kamera“ / AR-Technologie
→ Zeigt virtuelle Gegenstände von Schauspielern in Echtzeit
- umfangreiche Datensätze



Geometrisches Modellieren:

rechnergestützter Entwurf und Manipulation geometrischer Formen.

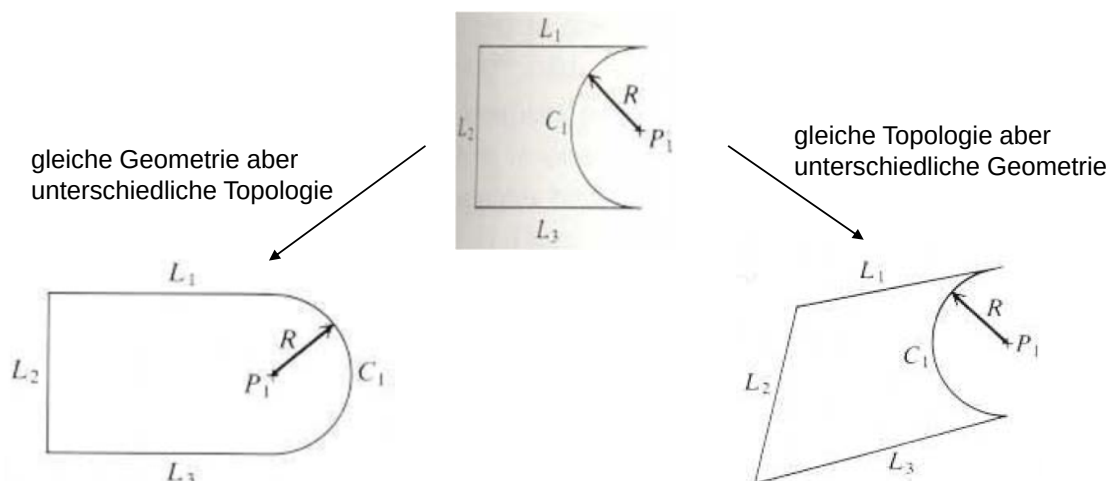
Methodik:

Beschreibung **geometrischer Formen** durch **Datenstrukturen**, die durch dazugehörige **Operatoren** manipuliert werden.

Verwendung der Darstellungen geometrischer Formen, die in der **Mathematik** entstanden sind, z.B. die Parameterdarstellung von Kurven und Flächen oder die Beschreibung eines geometrischen Objekts als Nullstellenmenge eines Gleichungs- oder Ungleichungssystems, die sogenannte implizite Darstellung.

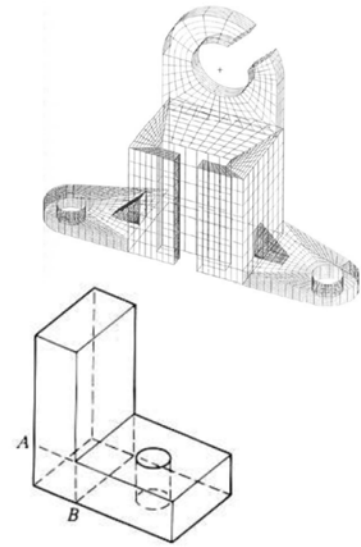
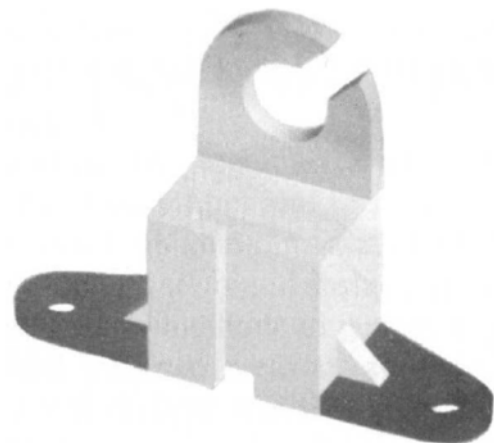
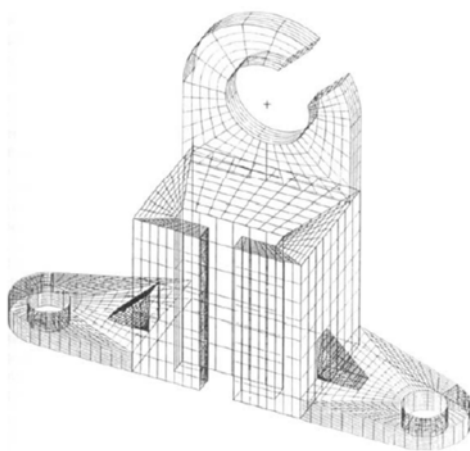
Repräsentation von Objekten einschließlich topologische und geometrischen Daten

- **Geometrie:** Form und Abmessungen
- **Topology:** Konnektivität und Assoziativität der Objekteinheiten; sie bestimmt die relationalen Informationen zwischen den Objekteinheiten



Grundlegende geometrische Modellierungstechniken

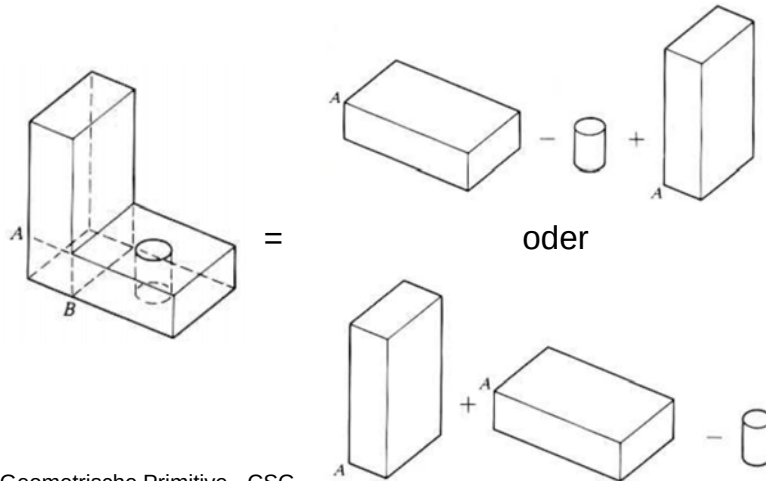
- 2D-Projektion (Zeichnungen)
- Drahtgitter-Modellierung
- **Surface Modeling** (Flächenmodellierung)
 - Analytische Fläche
 - Freiform, gekrümmte und skulpturale Oberfläche
- **Solid Modeling**
 - Konstruktive Volumenkörpergeometrie (CSG)
 - Boundary Repräsentation
 - Feature-basierte Modellierung
 - Parametrische Modellierung
 - usw.

**Surface Modeling** (Flächenmodellierung)

→ Flächenmodelle definieren nur die Geometrie, keine Topologie

Solid Modeling

Beispiele:



Volumen- und Randinformationen

Geometrische Primitive - CSG

Beispiele für Systeme:

Autodesk® **Maya**® www.autodesk.de/maya

Autodesk® **3ds Max**® www.autodesk.de/3dsmax



<http://www.blender.org/>



<http://www.opencascade.org>



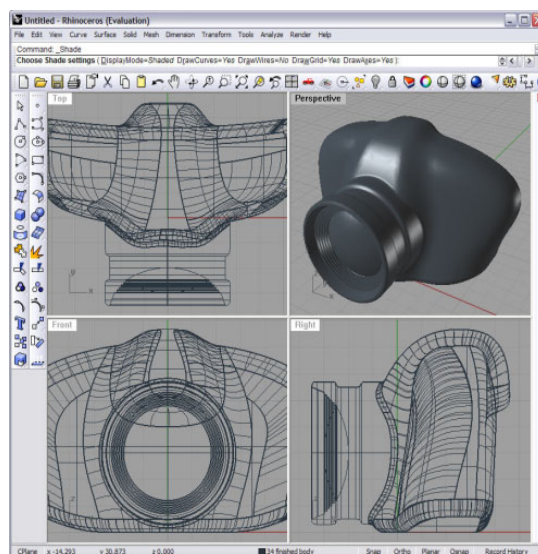
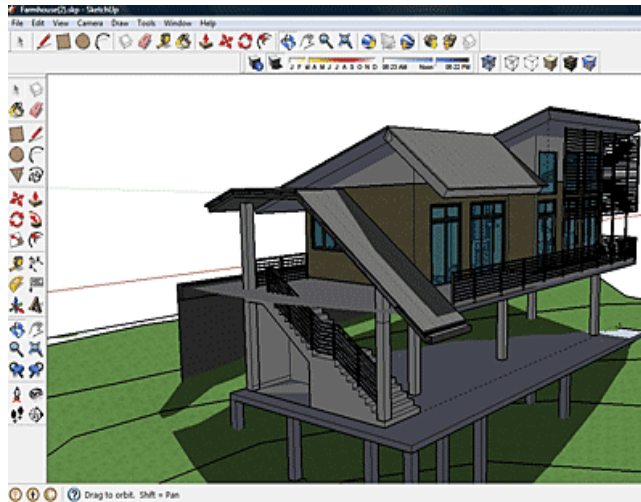
<http://www.rhino3d.com/>

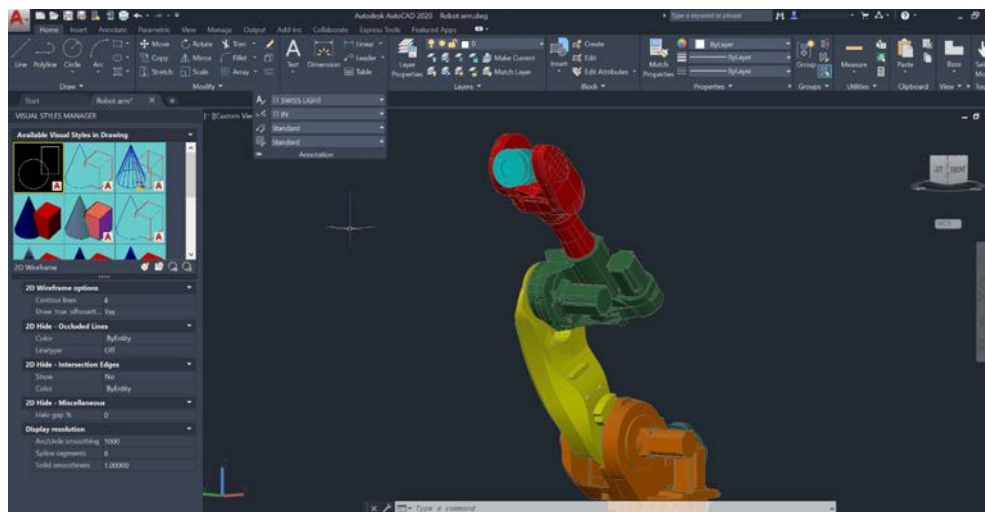


<http://sketchup.google.com/intl/de>



<http://sketchup.google.com/intl/de/>



**F.1. Einleitung**

- F.1.1. Motivation
- F.1.2. Grundkonzepte

F.2. Polynombasierte Repräsentationen

- F.2.1. Parameterdarstellung einer Kurve
- F.2.2. Implizite Darstellung einer Kurve
- F.2.3. Parameterdarstellung einer Fläche
- F.2.4. Explizite Darstellung von Flächen
- F.2.5. Implizite Darstellung von Flächen
- F.2.6. Polynomkurvensegmente
- F.2.7. Bézier-Kurvensegmente
- F.2.8. Bézier-Flächensegmente

F.3. Spline-basierte Repräsentation

- F.3.1. B-Spline-Funktionen
- F.3.2. Eigenschaften
- F.3.3. Knotenvektor
- F.3.4. Kontrollierbarkeit

F.4. Polygonbasierte Repräsentationen (Netze)

- F.4.1. Zellkomplexe (Netze)
- F.4.2. Datenstrukturen für Netze
- F.4.3. Polygonale Approximation von Kurven und Flächen
- F.4.4. Triangulierung
- F.4.5. Voronoi-Diagramm
- F.4.6. Delaunay-Triangulierung
- F.4.7. Netzverbesserung
- F.4.8. Netzreduktion


Parameterdarstellung einer Kurve im $\mathbb{R}^d$:

die d Koordinaten der Punkte $\mathbf{p}$ der Kurve ergeben sich als Funktionen über einem eindimensionalen Parameterbereich:

$$\mathbf{p} = \mathbf{k}(t), \quad \mathbf{k} : P \rightarrow \mathbb{R}^d, \quad t \in P \subseteq \mathbb{R}.$$

P ist ein Intervall, z.B. $[0,1]$, und heißt *Parameterbereich* der Kurve.

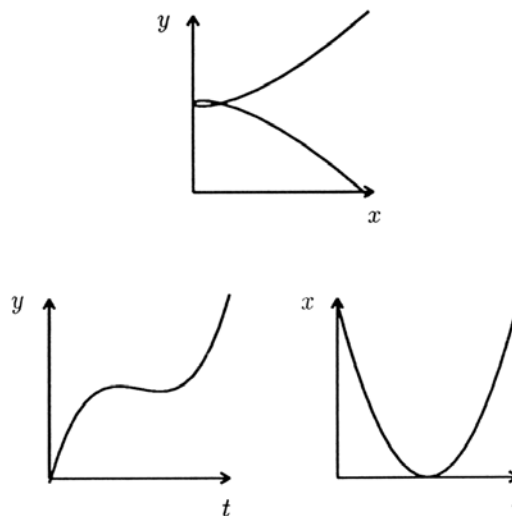
Beispiele:

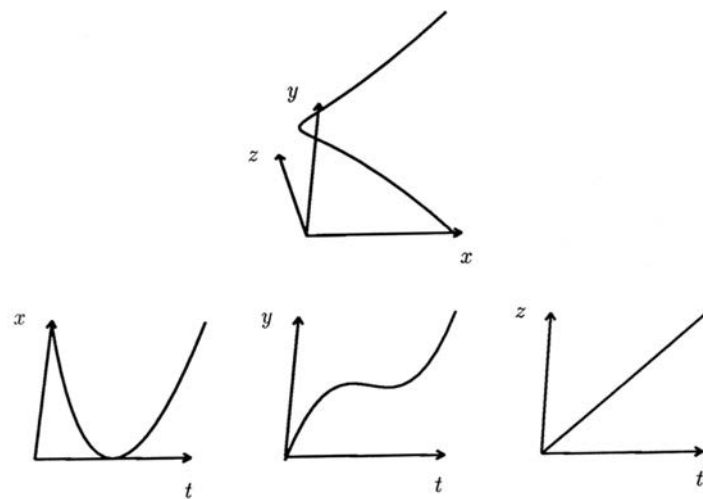
1. Kreis in der x - y -Ebene mit Mittelpunkt $\begin{pmatrix} m_x \\ m_y \end{pmatrix}$ und Radius r in trigonometrischer Parameterdarstellung:

$$x = m_x + r \cdot \cos t, \quad y = m_y + r \cdot \sin t, \quad t \in [0, 2\pi]$$

2. Eine Schraubenlinie im Raum lässt sich darstellen durch

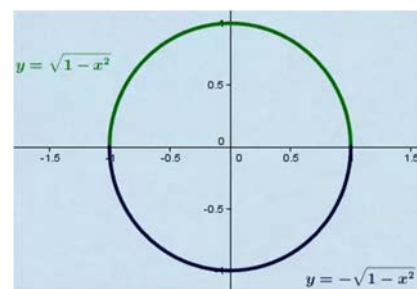
$$x = \sin t, \quad y = \cos t, \quad z = t, \quad t \in \mathbb{R}.$$


Bsp.: Ebene Kurve in Parameterdarstellung


Bsp.: Raumkurve in Parameterdarstellung**Beispiel: Repräsentation eines Kreises****1) explizite Darstellung:**Abbildung über zwei Funktionen

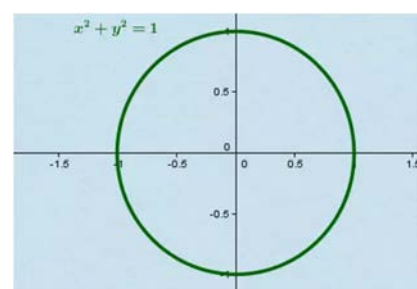
$$y = f(x) = \sqrt{1 - x^2}$$

$$y = -f(x) = -\sqrt{1 - x^2}$$

**2) implizite Darstellung:**Abbildung über eine Funktion

$$F(x, y) = 0$$

$$\sqrt{1 - x^2} - y = 0 \rightarrow x^2 + y^2 = 1$$



Implizite Darstellung einer Kurve im $\mathbb{R}^2$:

Nullstellenmenge einer Funktion $F: F(x, y) = 0$ mit $F: \mathbb{R}^2 \rightarrow \mathbb{R}$.

Beispiele:

1. Kreis in der x - y -Ebene mit Mittelpunkt $\begin{pmatrix} m_x \\ m_y \end{pmatrix}$ und Radius r :

$$(x - m_x)^2 + (y - m_y)^2 - r^2 = 0$$

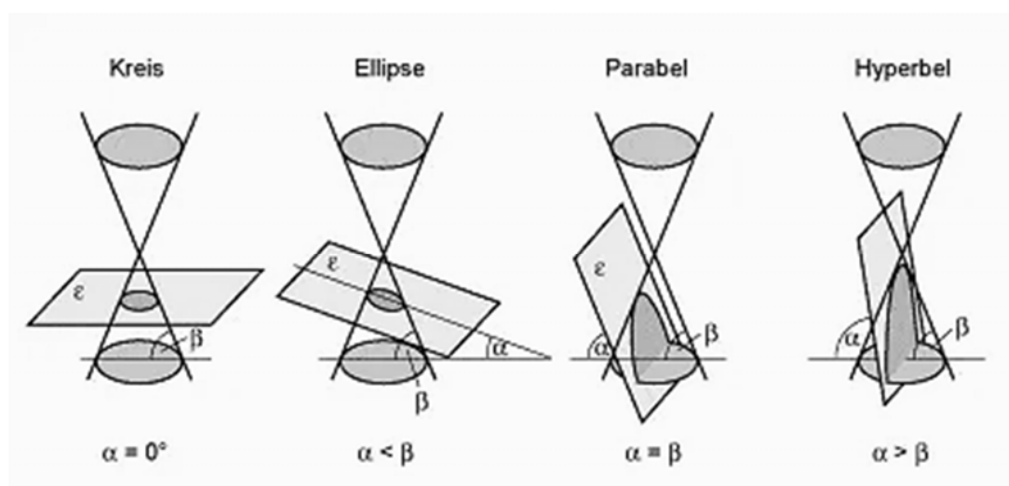
2. Kegelschnittkurven:

Schnittmenge einer Ebene mit einem Kegel

Formeldarstellung: $ax^2 + by^2 + cxy + dx + ey + f = 0$

Klassifikation der Kegelschnitte in Normaldarstellung:

- Ellipse: $x^2/a^2 + y^2/b^2 - 1 = 0$
- Hyperbel: $x^2/a^2 - y^2/b^2 - 1 = 0$
- Parabel: $y^2 - 2px = 0$.

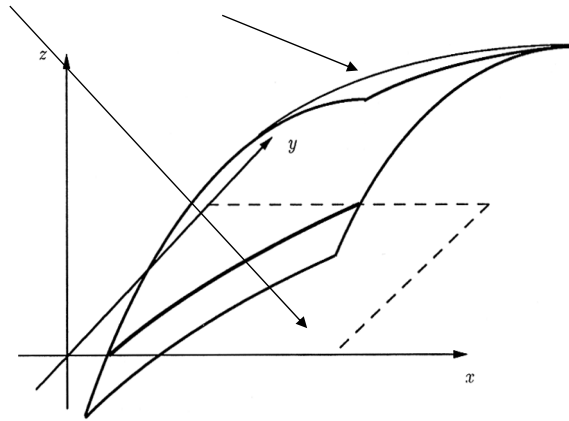
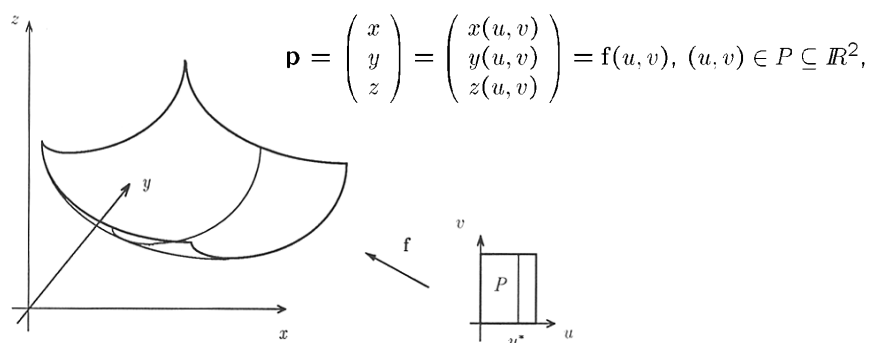
**Bemerkung:**

In impliziter Darstellung ist der Inzidenztest „Punkt auf Kurve“ besonders einfach, da lediglich die Punktkoordinaten in die Gleichung einzusetzen und der Funktionswert auf 0 zu testen ist.

Raumkurve in impliziter Darstellung:

der Schnitt zweier implizit definierter Flächen:

$$F(x, y, z) = 0, \quad G(x, y, z) = 0 \quad \text{mit} \quad F, G : \mathbb{R}^3 \rightarrow \mathbb{R}.$$

**Parameterdarstellung einer Fläche im $\mathbb{R}^3$:**

Die drei Koordinaten der Punkte $\mathbf{p}$ der Fläche ergeben sich als Funktionen über einem zweidimensionalen Parameterbereich P . P ist ein Intervall, beispielsweise $[0,1] \times [0,1]$, und heißt *Parameterbereich* der Fläche.

**Beispiel:**

$$x = \frac{u^2 + v^2}{3u^2 + 2v^2}, \quad y = \frac{u}{3u^2 + 2v^2}, \quad z = \frac{u \cdot v}{3u^2 + 2v^2}, \quad (u, v) \in (0, 1] \times (0, 1].$$

Bemerkung:

Hält man einen Parameter fest, beispielsweise $u = u^*$, dann beschreibt $\mathbf{f}(u^*, v)$ eine Kurve im Raum.

Die Schar der Kurven, die sich für beliebige u^* im Parameterbereich ergibt, legt die Fläche fest.

Natürlich kann analog auch v festgehalten werden.

**Explizite Darstellung einer Fläche im $\mathbb{R}^3$:**

die z-Komponente der Punkte der Fläche ergibt sich als Funktion der x- und y-Komponenten:

$$z = f(x, y) \text{ mit } f : P \rightarrow \mathbb{R}, P \subseteq \mathbb{R}^2.$$

Beispiel:

1. Sinuswelle im $\mathbb{R}^3$: $z = \sin x, x \in \mathbb{R}$,
2. x-y-Ebene im $\mathbb{R}^3$: $z = 0$.

Implizite Darstellung einer Fläche im $\mathbb{R}^3$:Nullstellenmenge einer Funktion F :

$$F(x, y, z) = 0 \text{ mit } F : \mathbb{R}^3 \rightarrow \mathbb{R}.$$

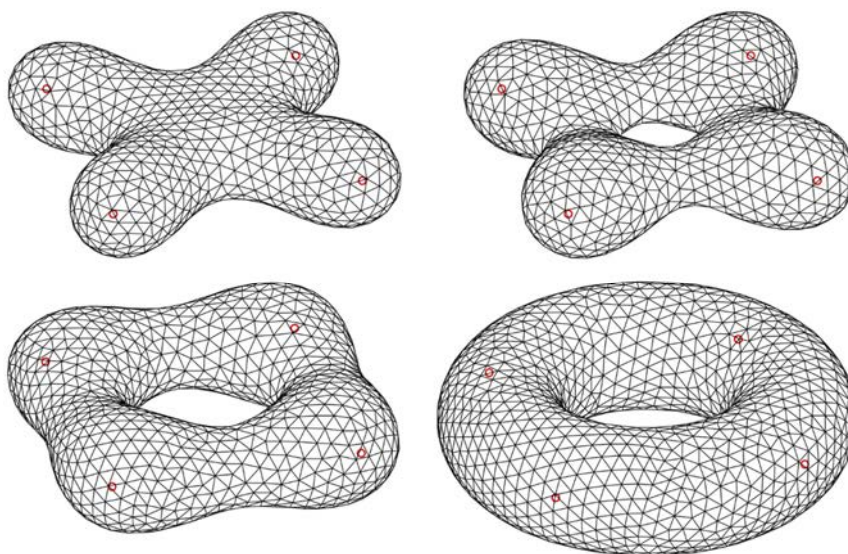
Beispiel:

- 1.
- Kugel:*

$$(x - m_x)^2 + (y - m_y)^2 + (z - m_z)^2 = r^2.$$

- 2.
- Quadriken allgemein:*

$$ax^2 + by^2 + cz^2 + dxy + exz + fyz + gx + hy + iz + j = 0,$$

Implizite Darstellungen von Flächen im $\mathbb{R}^3$:

Polynomkurvensegment in $\mathbb{R}^d$:

$$p(t) := \sum_{i=0}^n a_i t^i, \quad t \in [0, 1], \quad a_i \in \mathbb{R}^d.$$

Bemerkung:

Die Gestalt einer Polynomkurve in dieser Darstellung wird durch die Koeffizienten a_i bestimmt.

Problem:

Der Zusammenhang zwischen Kurve und Koeffizienten ist zu wenig intuitiv.

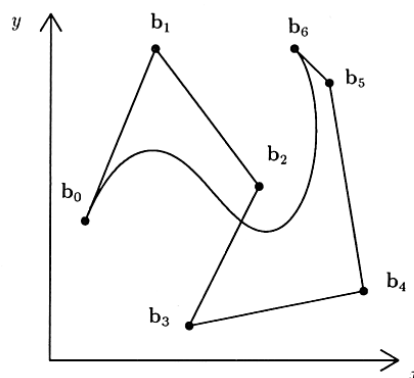
→ Lösung: *Bézier-Technik*

Bézier-Technik:

Bézier-Kurven sind gegeben durch eine Folge von *Bézier- Kontrollpunkten*

$$b_i \in \mathbb{R}^d, \quad i = 0, 1, \dots, n,$$

die einen sogenannten *Bézier-Kontrollpolygonzug* definieren.



Algorithmus von de Casteljau

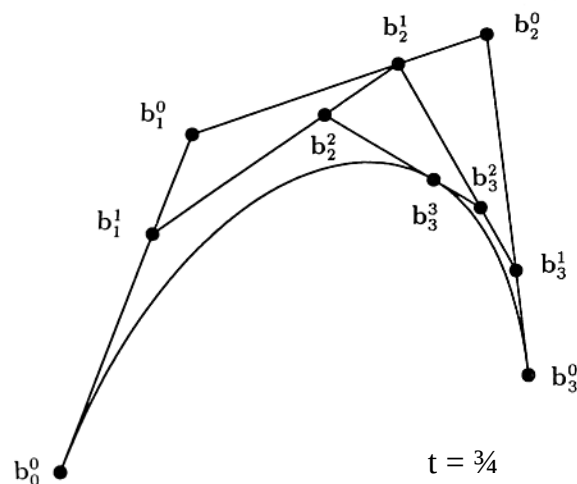
Eingabe: Bézier-Punkte $\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_n$, ein Parameterwert $t^* \in [0,1]$.

Ausgabe: Ein Kurvenpunkt $\mathbf{b}(t^*)$.

Ablauf:

```

BEGIN
  FOR  $i := 0$  TO  $n$  DO
     $\mathbf{b}_i^0 := \mathbf{b}_i$ ;
    FOR  $j := 1$  TO  $n$  DO;
      FOR  $i := j$  TO  $n$  DO;
         $\mathbf{b}_i^j := (1 - t^*) \cdot \mathbf{b}_{i-1}^{j-1} + t^* \cdot \mathbf{b}_i^{j-1}$ ;    (* Unterteilung *)
       $\mathbf{b}(t^*) := \mathbf{b}_n^n$ 
    END.
  
```

Berechnung der Kurvenpunkte: Algorithmus von de Casteljau

Satz (Formeldarstellung) Die durch den Algorithmus von de Casteljau definierte Kurve lässt sich formelmäßig durch

$$\mathbf{b}(t) := \sum_{i=0}^n \mathbf{b}_i B_i^n(t), \quad t \in [0, 1], \quad \mathbf{b}_i \in \mathbb{R}^d,$$

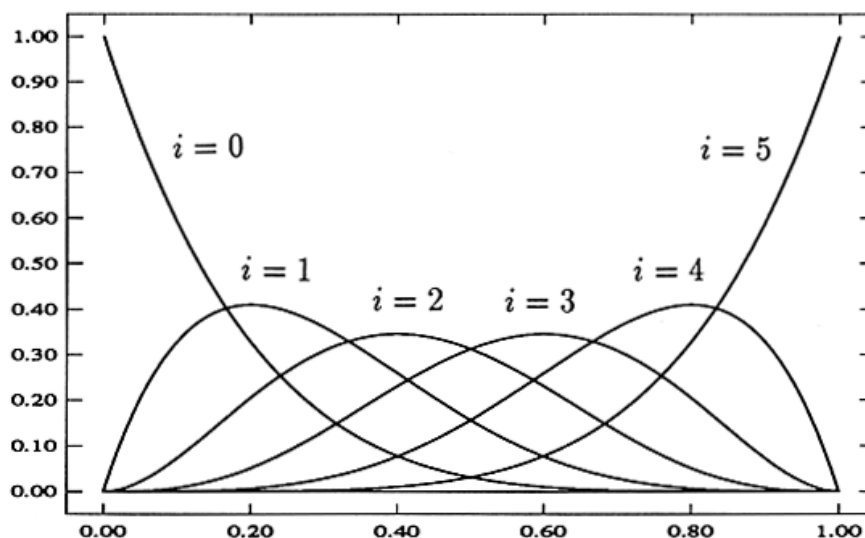
beschreiben, wobei die B_i^n die Bernstein-Polynome vom Grad n bezeichnen.

DEFINITION

Bernstein-Polynome vom Grad n :

$$B_i^n(x) = \binom{n}{i} x^i (1-x)^{n-i}, \quad i = 0, \dots, n.$$

Bsp.: Die Bernstein-Polynome vom Grad 5



Intuitive Herleitung der Formeldarstellung aus dem Algorithmus von de Casteljau:

$$\begin{aligned}
 b_1^1 &= (1-t) \cdot b_0^0 + t \cdot b_1^0 = b_0^0 \cdot B_0^1(t) + b_1^0 \cdot B_1^1(t) \\
 b_2^1 &= (1-t) \cdot b_1^0 + t \cdot b_2^0 = b_1^0 \cdot B_0^1(t) + b_2^0 \cdot B_1^1(t) \\
 b_3^1 &= (1-t) \cdot b_2^0 + t \cdot b_3^0 = b_2^0 \cdot B_0^1(t) + b_3^0 \cdot B_1^1(t) \\
 b_2^2 &= (1-t)^2 \cdot b_0^0 + 2t(1-t) \cdot b_1^0 + t^2 \cdot b_2^0 = b_0^0 \cdot B_0^2(t) + b_1^0 \cdot B_1^2(t) + b_2^0 \cdot B_2^2(t) \\
 b_3^2 &= (1-t)^2 \cdot b_1^0 + 2t(1-t) \cdot b_2^0 + t^2 \cdot b_3^0 = b_1^0 \cdot B_0^2(t) + b_2^0 \cdot B_1^2(t) + b_3^0 \cdot B_2^2(t) \\
 b_3^3 &= (1-t)^3 \cdot b_0^0 + 3t(1-t)^2 \cdot b_1^0 + 3t^2(1-t) \cdot b_2^0 + t^3 \cdot b_3^0 = b_0^0 \cdot B_0^3(t) + b_1^0 \cdot B_1^3(t) + b_2^0 \cdot B_2^3(t) + b_3^0 \cdot B_3^3(t)
 \end{aligned}$$

Allgemein: $b_k^j = \sum_{i=0}^j b_{k-j+i}^0 B_i^j(t)$ Kurvenpunkte: $b_n^n = \sum_{i=0}^n b_i^0 B_i^n(t)$

Formaler Beweis der Formel: durch Induktion über j (Übung)

Satz (Eigenschaften von Bernstein-Polynomen)

1. **Partition der 1:** Bernstein-Polynome summieren sich zu 1 auf:

$$\sum_{i=0}^n B_i^n(x) = \sum_{i=0}^n \binom{n}{i} x^i (1-x)^{n-i} = (x+1-x)^n = 1.$$

2. **Positivität:** Bernstein-Polynome sind über dem Einheitsintervall $[0,1]$ positiv:

$$B_i^n(x) \geq 0, \quad i = 0, \dots, n, \quad x \in [0, 1].$$

3. **Maxima:** $B_i^n(x)$ nimmt über $[0,1]$ sein Maximum in i/n an, $i = 0, \dots, n$.

4. Rekursion: Für $i = 0, 1, \dots, n-1$ gilt

$$\begin{aligned}
 B_{i+1}^{n+1}(x) &= \frac{(n+1)!}{(i+1)!(n-i)!} x^{i+1}(1-x)^{n-i} \\
 &= \left[\frac{n!(i+1)}{i!(n-i)!(i+1)} + \frac{n!(n-i)}{(i+1)!(n-i-1)!(n-i)} \right] x^{i+1}(1-x)^{n-i} \\
 &= x \cdot B_i^n(x) + (1-x) \cdot B_{i+1}^n(x).
 \end{aligned}$$

Resultierend somit *Bernstein-Polynome* vom Grad $n+1$ als *Konvexkombination* zweier *Bernstein-Polynome* vom Grad n .

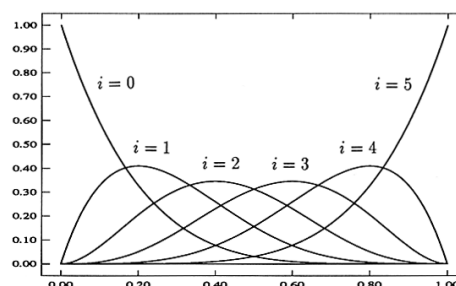
Beweis: Übung. $\square$

Satz (Eigenschaften von Bézier-Kurvensegmenten)

- 1. Verhalten in den Endpunkten:** Kontrollpolygon und Kurvensegment stimmen in Anfangs- und Endpunkt überein. Das erste und das letzte Segment des Kontrollpolygonzugs ist dort tangential zur Kurve. Die anderen Bézier-Punkte werden i.a. nicht interpoliert, sondern lediglich approximiert:

$$b(t) := \sum_{i=0}^n b_i B_i^n(t), \quad t \in [0, 1], \quad b_i \in \mathbb{R}^d.$$

Beweis über Eigenschaften der Bernstein-Polynome oder den de-Casteljau-Algorithmus



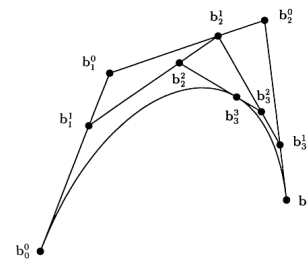
2. **Konvexe-Hülle-Eigenschaft:** Das Kurvensegment liegt in der konvexen Hülle der Bézier-Punkte. Es liegt damit auch in der konvexen Hülle des Bézier-Polygonzugs.



Eine Menge ist konvex, wenn mit je zwei Punkten auch deren Verbindungsstrecke in der Menge liegt.

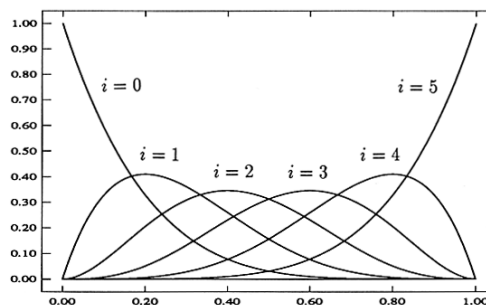
Die konvexe Hülle einer gegebenen Punktmenge A ist die kleinste konvexe Menge, die A umfasst.

Damit folgt die Konvexe-Hülle-Eigenschaft unmittelbar aus dem Algorithmus von de-Casteljau.



3. **Einfluss von Kontrollpunkten:**

$\mathbf{b}_i$ hat den größten Einfluss auf $\mathbf{b}(t)$ bei $t = i/n$.



4. **Beschränkte Schwankung:**

Keine Gerade schneidet ein Bézier-Segment öfter als den entsprechenden Bézier-Polygonzug. Anschaulich schwankt das Segment also nicht stärker als der Polygonzug (variation diminishing property).

Beweisidee später.

**Satz (Zeitaufwand des Algorithmus von de Casteljau)**

Der Zeitaufwand des Algorithmus von de Casteljau ist $O(n^2)$.

Bemerkung:

1. Mit einem Algorithmus, der vom Horner-Schema zur Auswertung von Polygonen abgeleitet ist, ist die Berechnung von Kurvenpunkten in $O(n)$ Zeit möglich.
2. Die geringere Effizienz des Algorithmus von de Casteljau wird in Situationen relativiert, wo aus den Zwischenergebnissen Nutzen gezogen wird, so wie es im Folgenden getan wird.
3. Der de-Casteljau-Algorithmus ist ferner numerisch recht stabil, da immer nur Konvexkombinationen bereits berechneter Daten ermittelt werden.

**Satz (Unterteilung von Bézier-Kurvensegmenten)**

Jeder Punkt $\mathbf{b}(t^*)$, $t^* \in [0,1]$, unterteilt ein Bézier-Segment in zwei Teilkurven. Diese Teilkurven sind wieder Bézier-Segmente gleichen Grades. Ihre Bézier-Punkte sind durch die Hilfspunkte

$$\mathbf{b}_0^0, \mathbf{b}_1^1, \dots, \mathbf{b}_n^n$$

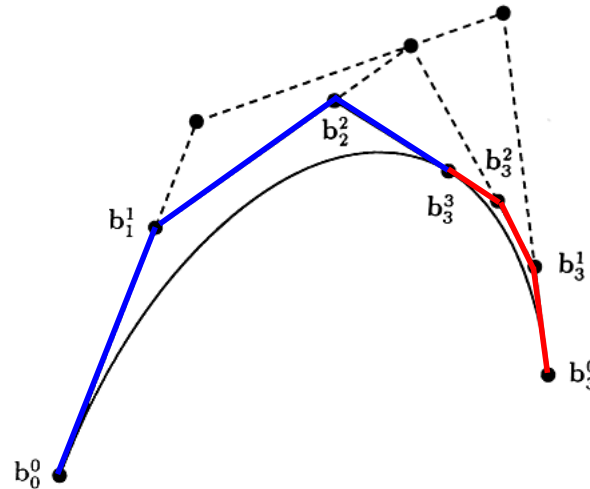
bzw.

$$\mathbf{b}_n^n, \mathbf{b}_{n-1}^{n-1}, \dots, \mathbf{b}_0^0$$

des Algorithmus von de Casteljau an der Stelle t^* gegeben.

Beweis: Übung. $\square$

Unterteilung von Bézier-Segmenten durch Ausführung des Algorithmus von de Casteljau



ALGORITHMUS (Unterteilung von Bézier-Kurvensegmenten)

unmittelbar aus dem vorigen Satz ableitbar.

Bemerkungen:

- Die konvexen Hüllen zu den Kontrollpunkten der beiden Teilsegmente werden im allgemeinen eine bessere Approximation als die ursprüngliche konvexe Hülle liefern.
- Iteriert man die Unterteilung mit $t^* = 1/2$, so resultiert ein zunehmend feinerer Polygonzug aus Bézier-Punkten zu Teilsegmenten, deren konvexe Hüllen die Kurve immer besser approximieren.

Im Grenzübergang konvergiert der Kontrollpolygonzug gegen die Kurve.

Diese Eigenschaft der Bézier-Segmente kann etwa bei der Entwicklung eines Algorithmus zum Schnitt zweier Bézier-Segmente verwendet werden.

Satz (Graderhöhung von Bézier-Kurvensegmenten)

Gegeben seien $n + 1$ Kontrollpunkte $b_0, b_1, \dots, b_n \in \mathbb{R}^d$ eines Bézier-Kurvensegments.

Das Bézier-Segment mit den $n + 2$ Kontrollpunkten $b_0^*, \dots, b_{n+1}^*$ mit

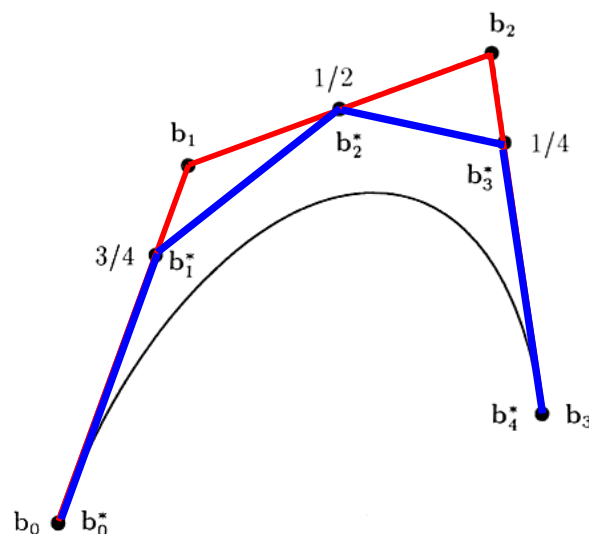
$$\begin{aligned} b_0^* &:= b_0, \\ b_i^* &:= \alpha_i b_{i-1} + (1 - \alpha_i) b_i, \quad \alpha_i := i/(n + 1), \quad i = 1, \dots, n, \\ b_{n+1}^* &:= b_n \end{aligned}$$

stimmt mit dem der $n + 1$ Kontrollpunkte überein, d.h. es gilt

$$\sum_{i=0}^n b_i B_i^n(t) = \sum_{i=0}^{n+1} b_i^* B_i^{n+1}(t).$$

Beweis: Übung. $\square$

Berechnung des neuen Kontrollpolygons bei Graderhöhung von Bézier-Segmenten:



ALGORITHMUS (Graderhöhung von Bézier-Kurvensegmenten)
unmittelbar aus dem vorigen Satz ableitbar.

Bemerkung:

Der Graderhöhungsalgorithmus erlaubt es, die beschränkte Schwankung von Bézier-Kurvensegmenten einzusehen:

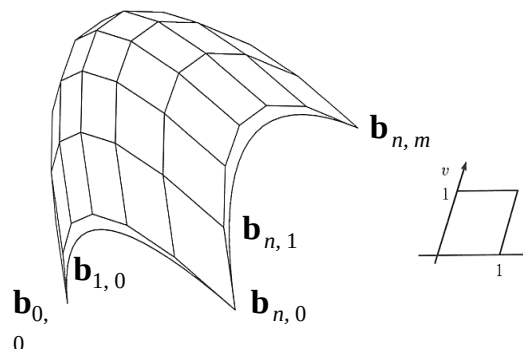
- Wählt man eine Folge von Stützstellen auf einer beliebigen Kurve, dann schneidet der Polygonzug, der durch deren Verbindung entsteht, eine beliebig vorgegebene Gerade nicht öfter als die gegebene Kurve.
- Im Falle von Bézier-Segmenten ist der Kontrollpolygonzug die gegebene Kurve. Der aus der Graderhöhung resultierende Polygonzug schneidet nach der obigen Beobachtung eine vorgegebene Gerade nicht öfter als der Ursprüngliche. Durch Iteration erhält man eine Folge von Polygonzügen, die gegen die Kurve konvergiert und für die jeder Polygonzug eine vorgegebene Gerade nicht öfter als sein Vorgänger schneidet. Dieses gilt daher auch für das Bézier-Kurvensegment.

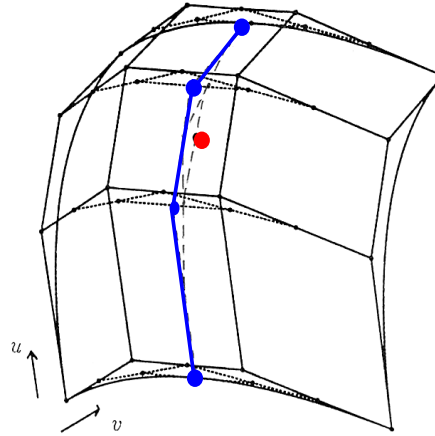
Bézier-Flächensegmente

Gegeben: $(n + 1) \cdot (m + 1)$ Bézier-(Kontroll-)Punkte $\mathbf{b}_{i,j}$, $i = 0, \dots, n$, $j = 0, \dots, m$.

Vierecks-Bézier-Segment:

$$\mathbf{b}(u, v) := \sum_{i=0}^n \sum_{j=0}^m \mathbf{b}_{i,j} \cdot B_i^n(u) \cdot B_j^m(v), \quad u, v \in [0, 1], \quad \mathbf{b}_{i,j} \in \mathbb{R}^3.$$




ALGORITHMUS (de Casteljau, für Vierecks-Bézier-Segmente)

ALGORITHMUS (de Casteljau, für Vierecks-Bézier-Segmente)

Eingabe: $(n + 1) \cdot (m + 1)$ Bézier-(Kontroll-)Punkte $\mathbf{b}_{i,j}$, $i = 0, \dots, n$,
 $j = 0, \dots, m$, $u^*, v^* \in [0, 1]$.

Ausgabe: Flächenpunkt $\mathbf{b}(u^*, v^*)$ des Flächenstücks zu den
 Bézier-Punkten $\mathbf{b}_{i,j}$, $i = 0, \dots, n$, $j = 0, \dots, m$.

1. Berechne $c_i := \sum_{j=0}^m \mathbf{b}_{i,j} \cdot B_j^m(v^*)$ nach dem Algorithmus von de Casteljau für Bézier-Kurvensegmente, $i = 0, \dots, n$.
2. Berechne $\mathbf{b}(u^*, v^*) := \sum_{i=0}^n c_i \cdot B_i^n(u^*)$ nach dem Algorithmus von de Casteljau für Bézier-Kurvensegmente.

**Satz (Zeitaufwand des Algorithmus von de Casteljau)**

Der Zeitaufwand des Algorithmus von de Casteljau für Vierecks-Bézier-Segmente ist $O(m^2 \cdot n + n^2)$, wenn zunächst in m -Richtung, dann in n -Richtung verfahren wird. Der Zeitaufwand mit dem Horner-Schema ist $O(m \cdot n + n)$.

Satz (Eigenschaften von Vierecks-Bézier-Segmenten)

1. **Konvexe-Hülle-Eigenschaft:** Das Flächenstück liegt in der konvexen Hülle des definierenden Kontrollnetzes.
2. **Einfluss von Kontrollpunkten:** Der Bézier-Punkt $\mathbf{b}_{i,j}$ hat den größten Einfluss auf das Segment bei $(u,v) = (i/n, j/m)$.
3. **Verhalten in Eckpunkten:** Die Eckpunkte des Kontrollnetzes und die Eckpunkte des Flächenstücks stimmen überein.
4. **Verhalten in Randkurven:** Die Randpunkte des Kontrollnetzes sind die Bézier-Punkte der Randkurven des Flächensegments.

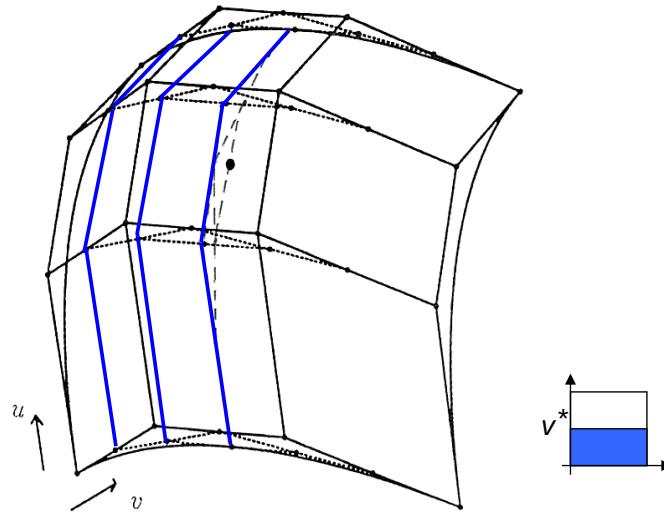


5. **Planarität:** Ein Bézier-Flächenstück ist genau dann eben, wenn das Bézier-Kontrollnetz ein ebenes Raster bildet.
6. **Flächenkurven:** Das Bild einer Geraden in der u - v -Ebene auf der Fläche ist eine Polynomkurve von höchstens dem Grad $m + n$.

Beweis: Übung.

Bemerkung:

Die Aussage über die beschränkte Schwankung lässt sich nicht unmittelbar von den Kurven auf die Flächen übertragen. Es ist nicht schwierig, ein Kontrollnetz und eine Gerade anzugeben, so dass zwar die Fläche durch die Gerade geschnitten wird, nicht aber das Kontrollnetz.

Unterteilung von Vierecks-Bézier-Segmenten:**Satz (Unterteilung von Vierecks-Bézier-Segmenten)**

Seien $\mathbf{b}_{i,j}^k$ die Punkte, die durch den Algorithmus von de Casteljau für die Kurven für $v = v^*$ geliefert werden. Die Vierecks-Bézier-Segmente

$$\mathbf{b}_l(u, v) = \sum_{i=0}^n \sum_{j=0}^m \mathbf{b}_{i,j}^j B_i^n(u) B_j^m(v), \quad u, v \in [0, 1],$$

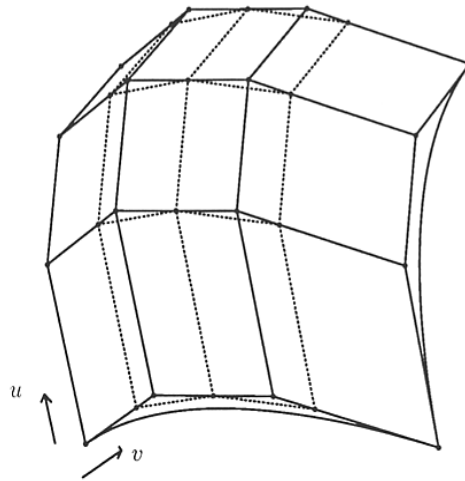
$$\mathbf{b}_r(u, v) = \sum_{i=0}^n \sum_{j=0}^m \mathbf{b}_{i,m}^{m-j} B_i^n(u) B_j^m(v), \quad u, v \in [0, 1],$$

zerlegen das Segment $\mathbf{b}(u, v) = \sum_{i=0}^n \sum_{j=0}^m \mathbf{b}_{i,j} B_i^n(u) B_j^m(v)$

in zwei längs der Kurve $\mathbf{b}(u, v^*)$ zusammengesetzte Teilsegmente.

ALGORITHMUS (Unterteilung von Vierecks-Bézier-Segmenten)

unmittelbar aus vorigem Satz herleitbar.

Graderhöhung von Vierecks-Bézier-Segmenten:**Satz (Graderhöhung von Vierecks-Bézier-Segmenten)**

Sei $\mathbf{b}(u, v) = \sum_{i=0}^n \sum_{j=0}^m \mathbf{b}_{i,j} B_i^n(u) B_j^m(v)$ ein Vierecks-Bézier-Segment.

Dann stimmt

$$\mathbf{b}^*(u, v) = \sum_{i=0}^n \sum_{j=0}^{m+1} \mathbf{b}_{i,j}^* B_i^n(u) B_j^{m+1}(v)$$

mit

$$\mathbf{b}_{i,0}^* := \mathbf{b}_{i,0},$$

$$\mathbf{b}_{i,m+1}^* := \mathbf{b}_{i,m},$$

$$\mathbf{b}_{i,j}^* := \alpha_j \mathbf{b}_{i,j-1} + (1 - \alpha_j) \mathbf{b}_{i,j}, \quad \alpha_j := j/(m+1), \quad j = 1, \dots, m,$$

mit der gegebenen Fläche $\mathbf{b}(u, v)$ überein.

ALGORITHMUS (Graderhöhung von Vierecks-Bézier-Segmenten)

unmittelbar aus vorigem Satz herleitbar.

F.1. Einleitung

- F.1.1. Motivation
- F.1.2. Grundkonzepte

F.2. Polynombasierte Repräsentationen

- F.2.1. Parameterdarstellung einer Kurve
- F.2.2. Implizite Darstellung einer Kurve
- F.2.3. Parameterdarstellung einer Fläche
- F.2.4. Explizite Darstellung von Flächen
- F.2.5. Implizite Darstellung von Flächen
- F.2.6. Polynomkurvensegmente
- F.2.7. Bézier-Kurvensegmente
- F.2.8. Bézier-Flächensegmente

F.3. Spline-basierte Repräsentation

- F.3.1. B-Spline-Funktionen
- F.3.2. Eigenschaften
- F.3.3. Knotenvektor
- F.3.4. Kontrollierbarkeit

F.4. Polygonbasierte Repräsentationen (Netze)

- F.4.1. Zellkomplexe (Netze)
- F.4.2. Datenstrukturen für Netze
- F.4.3. Polygonale Approximation von Kurven und Flächen
- F.4.4. Triangulierung
- F.4.5. Voronoi-Diagramm
- F.4.6. Delaunay-Triangulierung
- F.4.7. Netzverbesserung
- F.4.8. Netzreduktion

F.3. Spline-basierte Repräsentationen**F.3.1. B-Spline-Funktionen**

$$\mathbf{b}(t) := \sum_{i=0}^n \mathbf{b}_i B_i^n(t), \quad t \in [0, 1], \quad \mathbf{b}_i \in \mathbb{R}^d, \quad (\text{Bézier-Kurvensegment})$$

Nachteile der Bézier-Technik:

- **keine lokale Kontrollierbarkeit:** Die Veränderung eines Bézier-Punktes oder eines Gewichts bei der rationalen Form verändert das ganze Segment. Das erschwert die Feinabstimmung eines Entwurfs.
- **hoher Polynomgrad:** Bei komplexen Formen wird eine größere Zahl Bézier-Punkte benötigt. Mit jedem Bézier-Punkt erhöht sich der Grad um 1. Polynome hohen Grades erhöhen den Rechenaufwand und die Gefahr numerischer Ungenauigkeiten.

Lösung:

komplexe Formen aus einfacheren Teilen zusammensetzen, d.h. solchen von niedrigerem Grad

→ *Spline-Kurven*, speziell die *B-Spline-Kurven*

B-Splines (Cox-de Boor Rekursion)

$$P(t) = \sum_{i=1}^{n+1} B_i N_{i,k}(t) \quad t_{\min} \leq t \leq t_{\max} \quad 2 \leq k \leq n+1$$

$$\text{mit } N_{i,1}(t) = \begin{cases} 1 & \text{wenn } x_i \leq t \leq x_{i+1} \\ 0 & \text{sonst} \end{cases}$$

$$N_{i,k}(t) = \frac{(t - x_i) \cdot N_{i,k-1}(t)}{x_{i+k-1} - x_i} + \frac{(x_{i+k} - t) \cdot N_{i+1,k-1}(t)}{x_{i+k} - x_{i+1}}$$

beschreibt ein Spline-Funktion der Ordnung k (Support) mit

- Kontrollpunkten B_i (de Boor-Punkte)
- normalisierten B-Spline Basisfunktionen $N_{i,k}$
- Grad der Polynome ist $k-1$
- Elementen x_i des Knotenvektors mit $x_i \leq x_{i+1}$

Allgemeine Eigenschaften

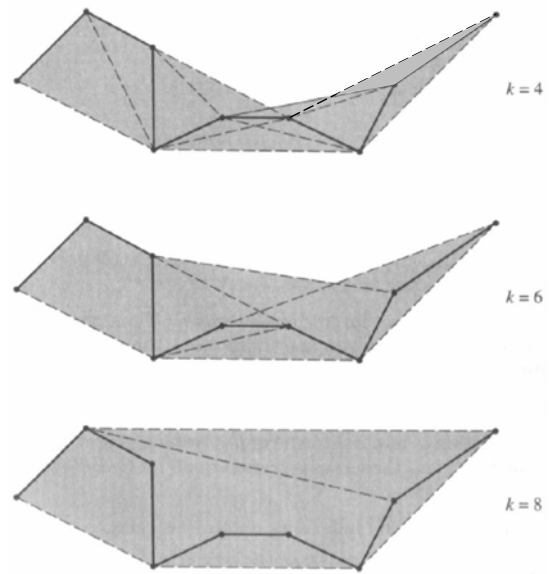
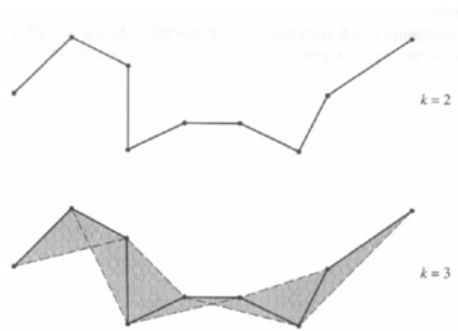
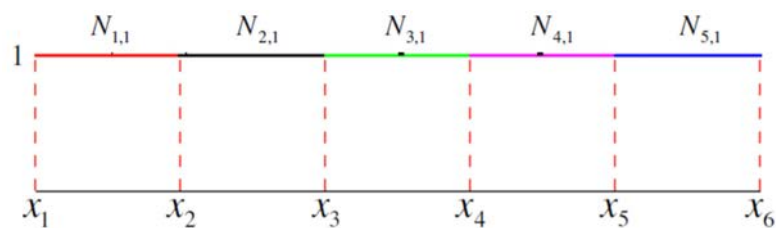
- $P(t)$ ist ein Polynom vom Grad $k-1$ in jedem Intervall
- $P(t)$ und seine Ableitungen $1, 2, \dots, k-2$ sind kontinuierlich über die gesamte Kurve
- Werte des Knotenvektors x_i beeinflussen die Basisfunktionen
- Basisfunktionen $N_{i,k}$ sind größer oder gleich 0.
- Summe der B-Spline Basisfunktionen für jeden Parameterwert t sind

$$\sum_{i=1}^{n+1} N_{i,k}(t) = 1$$

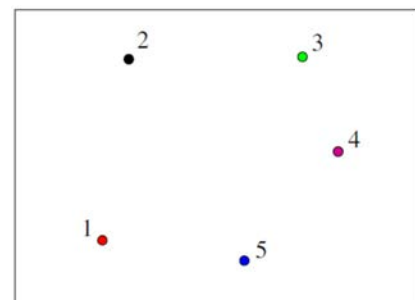
- Kurve liegt innerhalb der konvexen Hülle des Kontrollpolygons.
- maximale Ordnung der Kurve entspricht der Anzahl der
- Punkte des Kontrollpolygons, der Grad ist um Eins kleiner.

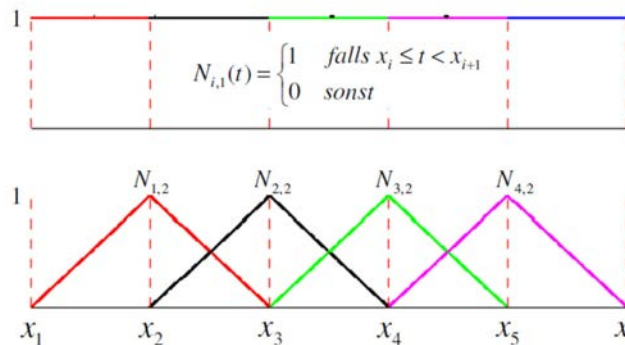
Konvexe Hüllen

→ abhängig von der Ordnung k

**Periodische B-Splines: $k = 1$** 

→ nur die Kontrollpunkte selbst



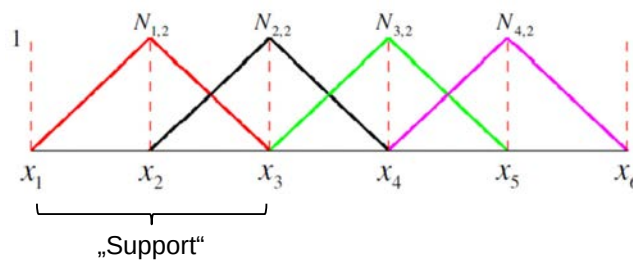
Periodische B-Splines: $k = 2$ 

Gewicht nimmt zum
Kontrollpunkt hin linear zu

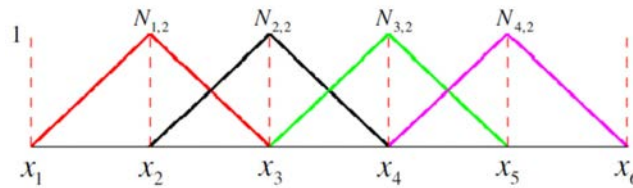
Gewicht nimmt vom
Kontrollpunkt weg linear ab

$$N_{i,2}(t) = \frac{(t - x_i) \cdot N_{i,1}(t)}{x_{i+1} - x_i} + \frac{(x_{i+2} - t) \cdot N_{i+1,1}(t)}{x_{i+2} - x_{i+1}}$$

Normierung: Division durch
Fläche unter Basisfunktion

Periodische B-Splines: $k = 2$ 

- $k = 2$: ein Kontrollpunkt beeinflusst 2 Intervalle, bzw. in jedem Intervall beeinflussen 2 Punkte
- Grad des Polynoms: 1 (stückweise linear)
- $n = 4$: somit 4 Kontrollpunkte und 4 Basisfunktionen
- Knotenvektor besteht aus 6 Knoten (4 „Doppel“-Intervalle)

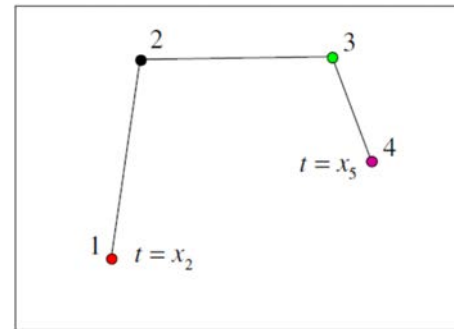
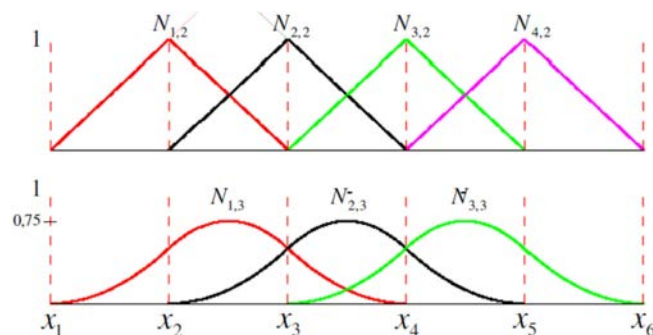
Periodische B-Splines: $k = 2$ 

Bedingung $\sum_{i=1}^{n+1} N_{i,k}(t) = 1$

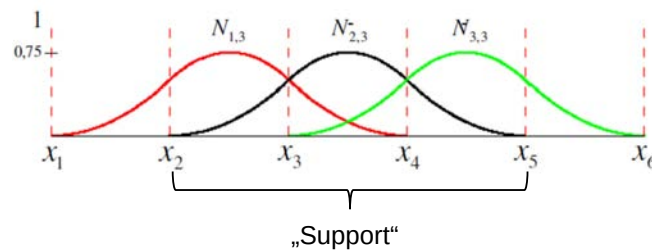
ist nur erfüllt im Bereich

$$x_2 \leq t < x_5$$

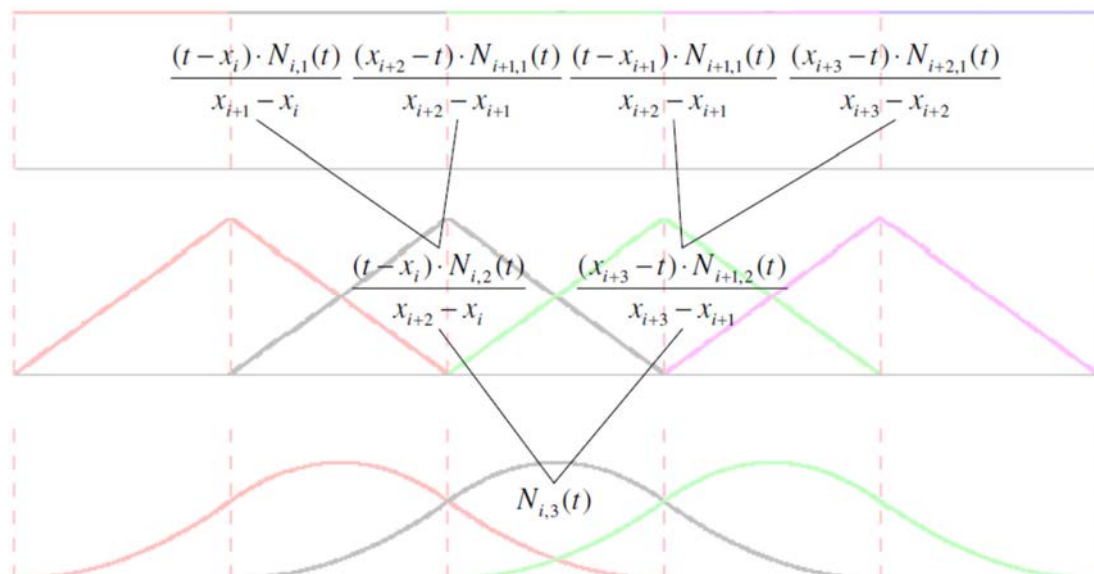
→ Kurve nur in diesem Parameterbereich gezeichnet werden.

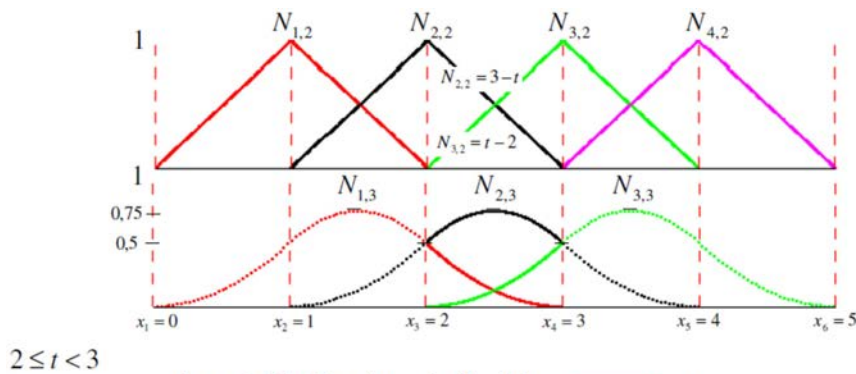
Periodische B-Splines: $k = 3$ 

$$N_{i,3}(t) = \frac{(t - x_i) \cdot N_{i,1}(t)}{x_{i+2} - x_i} + \frac{(x_{i+3} - t) \cdot N_{i+1,2}(t)}{x_{i+3} - x_{i+1}}$$

Periodische B-Splines: $k = 3$ 

- $k = 3$: Support für 3 Intervalle
- Grad des Polynoms: 2 (stückweise Parabeln)
- Knotenvektor 6 Knoten (3 „Tripel“-Intervalle)
- Gültiger Parameterbereich für Kurve: $x_3 \leq t < x_4$

Periodische B-Splines: $k = 3$ (Rekursive Definition)

Periodische B-Splines: $k = 3$  $2 \leq t < 3$

$$N_{1,3}(t) = \frac{(t-x_1) \cdot N_{1,2}(t)}{x_3-x_1} + \frac{(x_4-t) \cdot N_{2,2}(t)}{x_4-x_2}$$

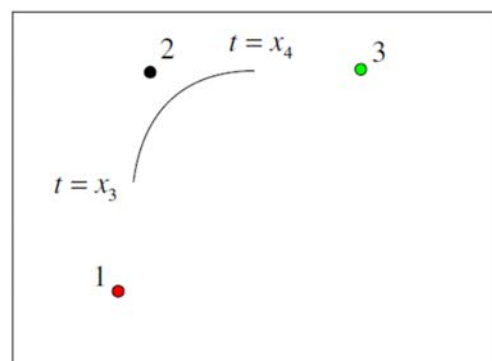
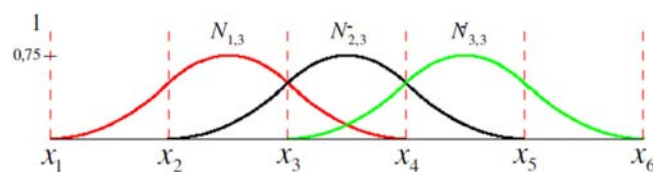
$$N_{1,3}(t) = \frac{1}{2}(3-t)^2$$

$$N_{2,3}(t) = \frac{(t-x_2) \cdot N_{2,2}(t)}{x_4-x_2} + \frac{(x_5-t) \cdot N_{3,2}(t)}{x_5-x_3}$$

$$N_{2,3}(t) = \frac{1}{2}((t-1) \cdot (3-t) + (4-t) \cdot (t-2))$$

$$N_{3,3}(t) = \frac{(t-x_3) \cdot N_{3,2}(t)}{x_5-x_3} + \frac{(x_6-t) \cdot N_{4,2}(t)}{x_6-x_4}$$

$$N_{3,3}(t) = \frac{1}{2}(t-2)^2$$

stückweise
ParabelnPeriodische B-Splines: $k = 3$ 

Knotenvektoren

- einzige Bedingung für den Knotenvektor ist $x_i \leq x_{i+1} \in \mathbb{R}$
→ es wird lediglich Monotonie gefordert, keine strenge Monotonie.
- Für $x_{i+k-1} - x_i = 0$ wird die Division durch 0 als 0 definiert.
- Je nach Wahl des Knotenvektors wird unterscheiden zwischen
 - a) offener Knotenvektor**
 - b) uniformer (gleichmäßiger) Knotenvektor**
 - c) periodischer Knotenvektor**
- **Nichtuniforme Knotenvektoren** haben ungleiche Abstände und/oder Mehrfachknoten, z.B.

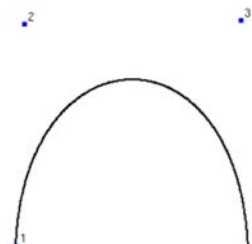
[0 0 0 1 1 2 2 2]	offen
[0 1 2 2 3 4]	periodisch
[0 0,28 0,5 0,72 1]	periodisch

Knotenvektoren

- **Nichtuniforme Knotenvektoren** haben ungleiche Abstände und/oder Mehrfachknoten, z.B.

[0 0 0 1 1 2 2 2]	offen
[0 1 2 2 3 4]	periodisch
[0 0,28 0,5 0,72 1]	periodisch
- Wenn die Ordnung einer B-Spline Kurve gleich der Anzahl der Kontrollpunkte ist ($n=k$), dann reduziert die B-Spline Basis zur Bernstein-Basis → **B-Spline Kurve wird zur Bézier Kurve**

Beispiel:

 $n = 4, k = 4$, Knotenvektor = [0 0 0 0 1 1 1 1]

Uniforme Knotenvektor

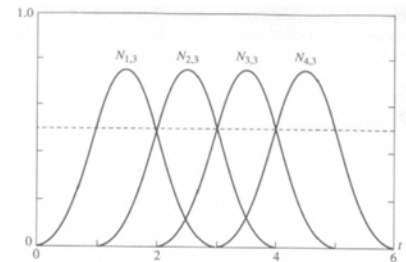
- Uniforme Knotenvektoren sind gleichabständig

Beispiele: $[0 \ 1 \ 2 \ 3 \ 4]$ oder $[-0,2 \ -0,1 \ 0 \ 0,1 \ 0,2]$

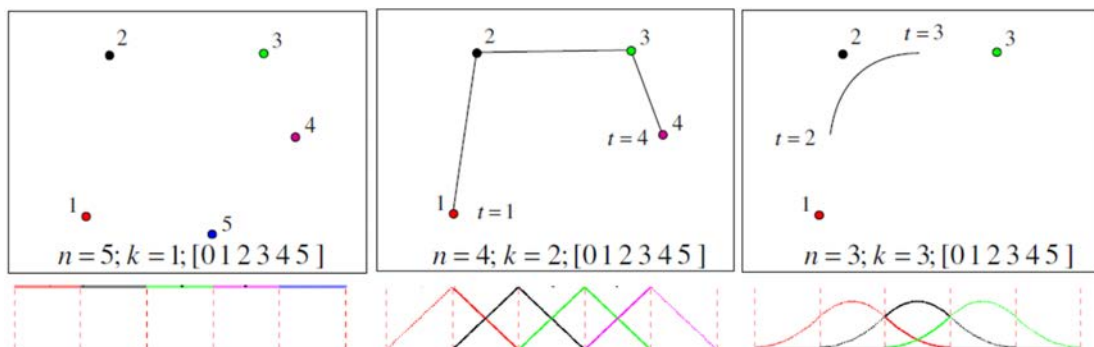
$[0 \ 0,25 \ 0,5 \ 0,75 \ 1]$ → zusätzlich normalisierte Form

- Für **periodische uniforme** Knotenvektoren ergibt sich

$$N_{i,k}(t) = N_{i-1,k}(t-1) = N_{i+1,k}(t+1)$$



$[x]=[0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6]$ $n+1=4$ $k=3$

Periodische uniforme Splines

- Anzahl der Knoten im Knotenvektor: $n + k$
- Gültiger Parameterbereich: $k - 1 \leq t \leq n$
- max. Ordnung bei n Kontrollpunkten: $k = n$
- Knotenvektor: $[0 \ 1 \ 2 \ \dots \ n + k - 1]$

offene Knotenvektor

- Knotenvektoren haben k Mehrfachknoten an den Enden, interne Knoten sind gleichabständig

Beispiel: $[0\ 0\ 0\ 1\ 2\ 3\ 3\ 3]$ für $k = 3$

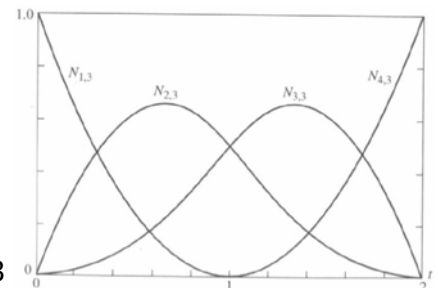
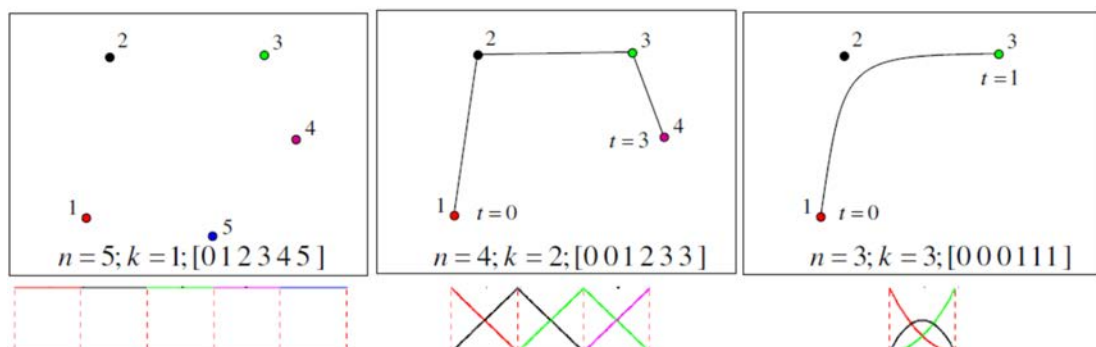
- Ein Spline der Ordnung k mit n Kontrollpunkte hat $n+k$ Knotenvektor
- allgemein

$$x_i = 0 \quad \text{für} \quad 1 \leq i \leq k$$

$$x_i = i - k \quad \text{für} \quad k + 1 \leq i \leq n + 1$$

$$x_i = n - k + 2 \quad \text{für} \quad n + 2 \leq i \leq n + k + 1$$

$$[x] = [0\ 0\ 0\ 1\ 2\ 2\ 2] \quad n+1=4 \quad k=3$$

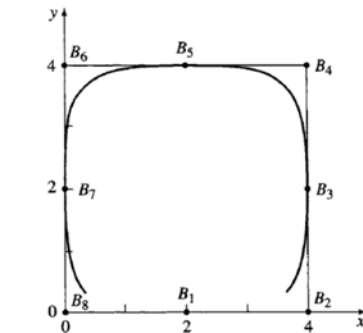
**Offene uniforme Splines/Knotenvektoren**

- Anzahl der Knoten im Knotenvektor: $n + k$
- Gültiger Parameterbereich: $0 \leq t \leq n - k + 1$
- max. Ordnung bei n Kontrollpunkten: $k = n$
- Knotenvektor: $[\underbrace{0 \dots 0}_k \ 1 \ 2 \ 3 \ \dots \ \underbrace{3}_k]$

Geschlossene Splines

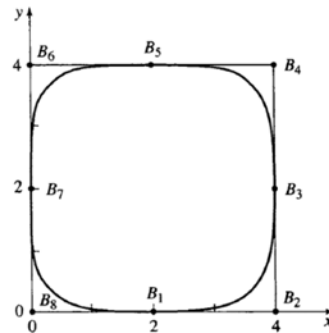
- Hierfür sind die periodischen Knotenvektoren sinnvoll
- Wiederholung von insgesamt $k - 2$ Kontrollpunkten am Anfang oder/und Ende schließt die Kurve

Beispiel: $k=4$, 8 Kontrollpunkte



Kontrollpolygon: $B_1 B_2 B_3 B_4 B_5 B_6 B_7 B_8 B_1$
 Knotenvektor: $[0 \ 1 \ 2 \ \dots \ 10 \ 11 \ 12]$

→ B-Spline **nicht** geschlossen



Kontrollpolygon: $B_8 B_1 B_2 B_3 B_4 B_5 B_6 B_7 B_8 B_1 B_2$
 Knotenvektor: $[0 \ 1 \ 2 \ \dots \ 12 \ 13 \ 14]$

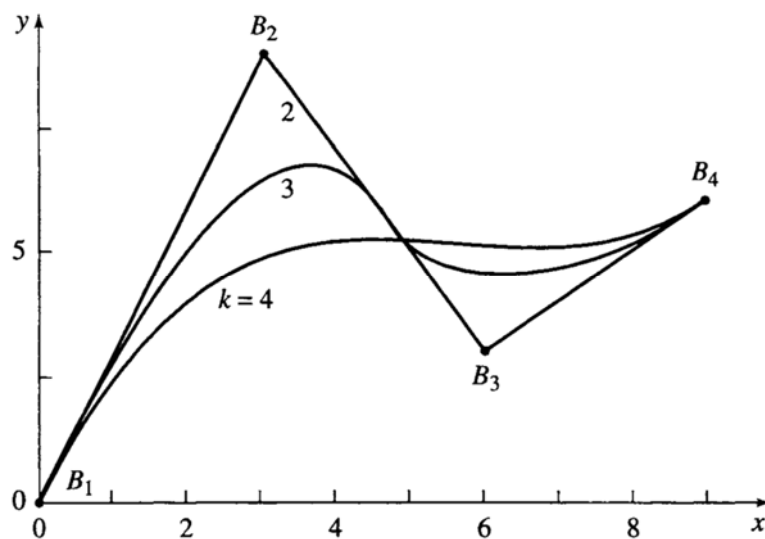
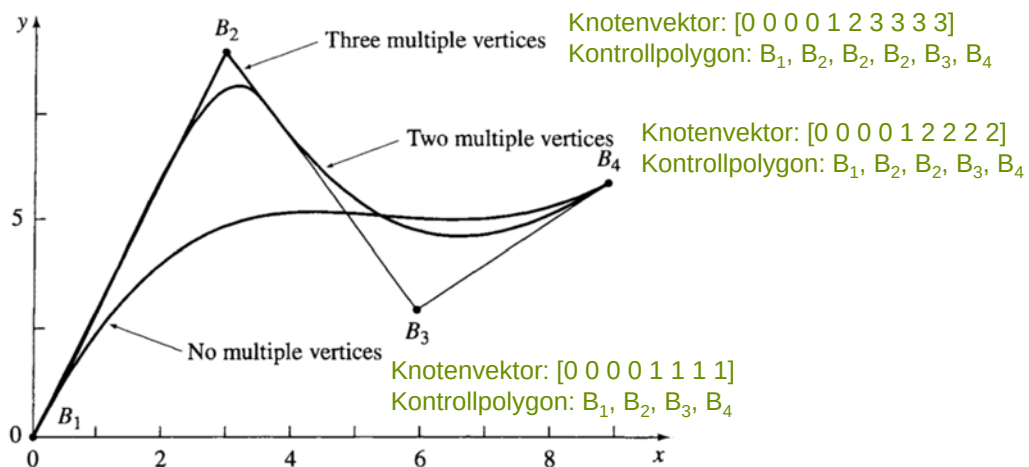
→ B-Spline **geschlossen**

Kontrollierbarkeit einer B-Spline Kurve

- Ändern des Typs der Basisfunktion
periodisch uniform, offen uniform, nichtuniform
- Ändern der Ordnung k der Basisfunktionen
- Ändern der Zahl und/oder der Position der Punkte des Kontrollpolygons
- Mehrfache Kontrollpunkte
- Mehrfachwerte im Knotenvektor

Bemerkung:

Die B-Spline-Technik lässt sich mit der gleichen Konstruktion wie bei Vierecks-Bézier-Segmenten auf Flächen ausweiten. Diese wird auch als *Tensorproduktkonstruktion* bezeichnet.

Kontrollierbarkeit einer B-Spline KurveBeispiel: **Ändern der Ordnung k der Basisfunktionen****Kontrollierbarkeit einer B-Spline Kurve**Beispiel: **Mehrfache Kontrollpunkte ($k=4$)**

Literatur

- S. Abramowski, H. Müller, Geometrisches Modellieren, BI-Wissenschaftsverlag, 1991 (vergriffen, als pdf-Datei auf der Vorlesungsseite verfügbar)
- David F. Rogers, An Introduction to NURBS, Morgan Kaufmann, 2007
- Parent, Rick, Computer Animation: Algorithms and Techniques, Elsevier, 2012
- J. Hoschek, D. Lasser, Grundlagen der geometrischen Datenverarbeitung, Teubner-Verlag, 1992

F.1. Einleitung

- F.1.1. Motivation
- F.1.2. Grundkonzepte

F.2. Polynombasierte Repräsentationen

- F.2.1. Parameterdarstellung einer Kurve
- F.2.2. Implizite Darstellung einer Kurve
- F.2.3. Parameterdarstellung einer Fläche
- F.2.4. Explizite Darstellung von Flächen
- F.2.5. Implizite Darstellung von Flächen
- F.2.6. Polynomkurvensegmente
- F.2.7. Bézier-Kurvensegmente
- F.2.8. Bézier-Flächensegmente

F.3. Spline-basierte Repräsentation

- F.3.1. B-Spline-Funktionen
- F.3.2. Eigenschaften
- F.3.3. Knotenvektor
- F.3.4. Kontrollierbarkeit

F.4. Polygonbasierte Repräsentationen (Netze)

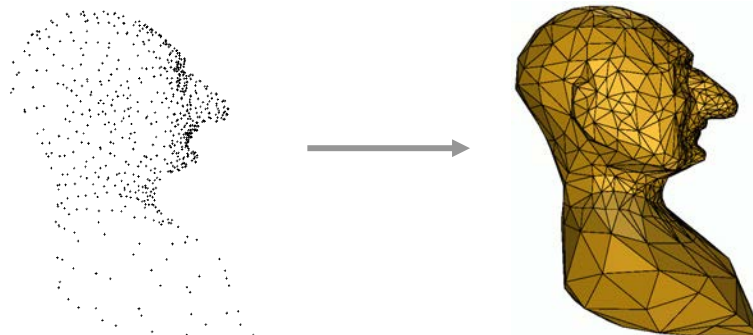
- F.4.1. Zellkomplexe (Netze)
- F.4.2. Datenstrukturen für Netze
- F.4.3. Polygonale Approximation von Kurven und Flächen
- F.4.4. Triangulierung
- F.4.5. Voronoi-Diagramm
- F.4.6. Delaunay-Triangulierung
- F.4.7. Netzverbesserung
- F.4.8. Netzreduktion



Bsp.:

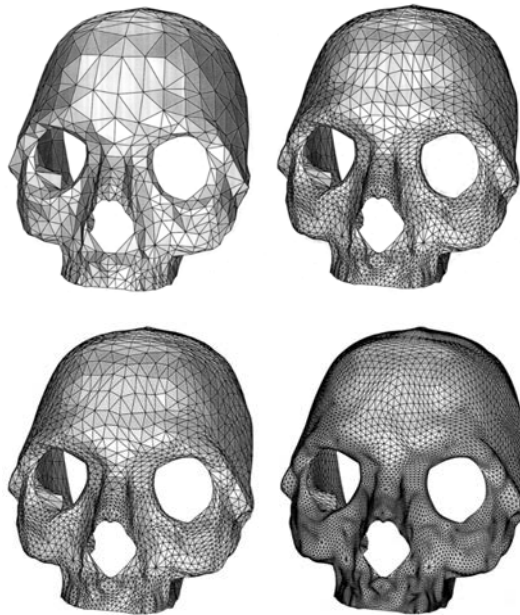


Bsp.:





Bsp.:



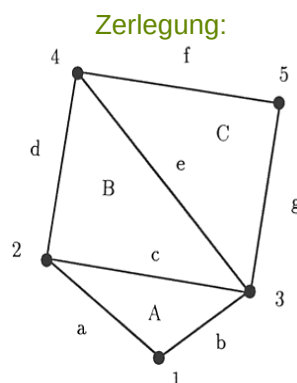
Verfahren zur Speicherung von Netzen:

- Zellinzidenzgraph
- Winged-Edge-Datenstruktur
- Halbkantendatenstruktur (Half-Edge Data Structure)
- Facetten-Knoten-Inzidenzliste

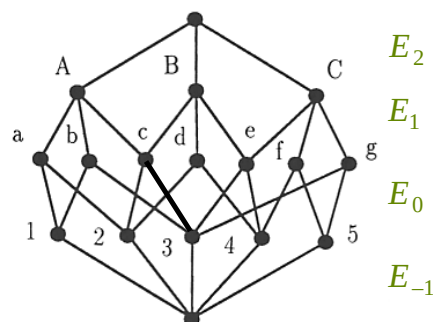
Zellinzidenzgraph:

- ein Graph $I = (V, E)$ für eine Zerlegung im $\mathbb{R}^d$.
- Die Knotenmenge V ist die disjunkte Vereinigung aus Teilmengen V_i , $i = -1, \dots, d+1$.
 V_i , $i = 0, \dots, d$, enthält für jede i -dimensionale Zelle einen Knoten. V_{-1} und V_{d+1} enthalten genau einen Knoten.
- Die Kantenmenge E ist die disjunkte Vereinigung aus Teilmengen E_i , $i = -1, \dots, d$. E_i enthält Kanten zwischen Knoten in V_i und V_{i+1} .
 Ein Knoten $v \in V_i$ ist mit einem Knoten $w \in V_{i+1}$ genau dann mit einer Kante verbunden, wenn v eine Randzelle von w ist. Der Knoten in V_{-1} ist mit allen Knoten in V_0 verbunden, der Knoten in V_{d+1} mit allen Knoten in V_d .

H. Edelsbrunner, Algorithms in Combinatorial Geometry, Springer-Verlag, 1987

Bsp.: Zellinzidenzgraph

entsprechender Zellinzidenzgraph:



$$V_0 = \{1, 2, \dots, 5\}, V_1 = \{a, b, \dots, g\}, V_2 = \{A, B, C\}.$$

Die Menge E_1 der zwischen V_1 und V_2 verlaufenden Kanten ist

$$E_1 = \{\{a, A\}, \{b, A\}, \{c, A\}, \{c, B\}, \{d, B\}, \{e, B\}, \{e, C\}, \{f, C\}, \{g, C\}\}.$$

Entsprechend können E_{-1} , E_0 und E_2 dargestellt werden.

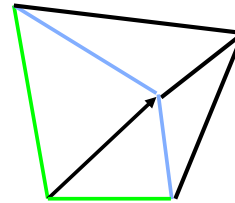
**Winged-Edge-Struktur:**

eine verzweigte Struktur aus drei Typen von Klassen, je eine für Knoten, Kanten und Gebiete:

```
class Vertex {
    Edge Nachbarkanten;
    Koordinatentyp Koordinaten; }
```

```
class Edge {
    Vertex VStart, VEnde;
    Face FRechts, FLinks;
    Edge NERechts, PERechts, NELinks, PELinks; }
```

```
class Face {
    Edge Nachbarkanten; }
```

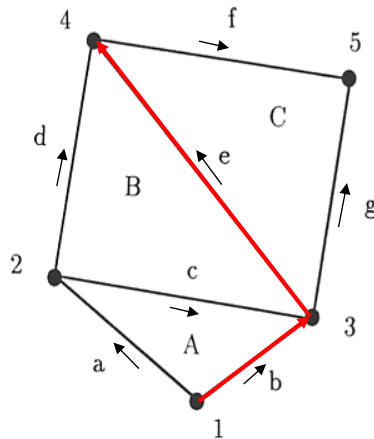
**Winged-Edge-Struktur:**

1. VStart, VEnde: Verweise auf die beiden Endknoten der Kante.
2. FRechts, FLinks: Verweis auf die höchstens zwei Nachbargebiete der Kante. Links und rechts gilt dabei bezüglich des Kantendurchlaufs vom Anfangs- zum Endpunkt.
3. PERechts, PELinks: die höchstens zwei Vorgänger der Kante.
4. NERechts, NELinks: die höchstens zwei Nachfolger der Kante.
5. In jedem Punkt-Objekt wird außer den Koordinaten noch der Verweis auf irgendeine inzidente Kante abgelegt.
In den Gebiets-Objekten ist analog eine inzidente Gebietskante abgelegt.
6. Über die in den Kanten-Objekten verfügbaren Nachbarkanten können durch Weiterhangeln über Kanten-Objekte alle zu einem Eckpunkt bzw. einem Gebiet inzidenten Kanten mit linearem Zeitaufwand gefunden werden.

K. Weiler, Edge-based Data Structures for Solid Modeling in Curved-Surface Environments, IEEE Computer Graphics and Applications 5(1) 1985, 21-40

Winged-Edge-Struktur

Beispiel:



Fläche	A	B	C
Nachbarkanten	c	e	e

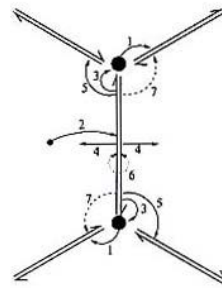
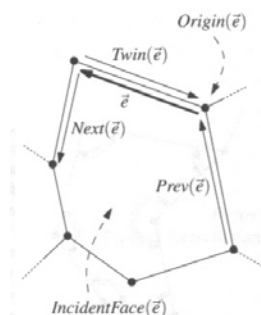
Knoten	1	2	3	4	5
Nachbarkanten	a	d	c	d	f
Koordinaten	x1	x2	x3	x4	x5
	y1	y2	y3	y4	y5

Kante	a	b	c	d	e	f	g
VStart	1	1	2	2	3	4	3
VEnde	2	3	3	4	4	5	5
FRechts	A	-	A	B	C	C	-
FLinks	-	A	B	-	B	-	C
NERechts	c	g	b	e	f	g	f
NELinks	d	c	e	f	d	g	f
PERechts	b	a	a	c	g	e	b
PELinks	b	a	d	a	c	d	e

Bemerkung:

Reduzierte Version der Winged-Edge-Datenstruktur:

Halbkantendatenstruktur



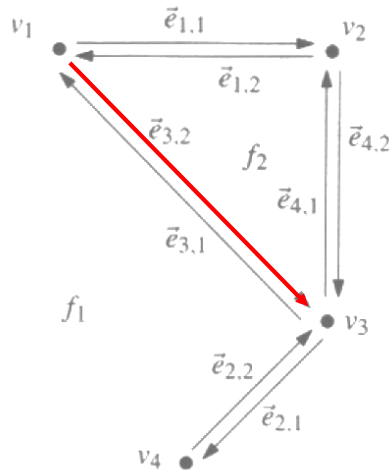
1. Vertex $\mapsto$ one outgoing halfedge,
2. Face $\mapsto$ one halfedge,
3. Halfedge $\mapsto$ target vertex,
4. Halfedge $\mapsto$ its face,
5. Halfedge $\mapsto$ next halfedge,
6. Halfedge $\mapsto$ opposite halfedge (implicit),
7. Halfedge $\mapsto$ previous halfedge (optional).

LITERATUR:

M. Botsch, S. Steinberg, S. Bischoff, L. Kobbelt,
 OpenMesh - a generic and efficient polygon mesh data structure,
 OpenSG Forum, 2002 - Online documentation of the OpenMesh package:
<http://www-i8.informatik.rwth-aachen.de>, research section

Halbkantendatenstruktur

Beispiel:



Vertex	Coordinates	IncidentEdge
v_1	(0,4)	$\vec{e}_{1,1}$
v_2	(2,4)	$\vec{e}_{4,2}$
v_3	(2,2)	$\vec{e}_{2,1}$
v_4	(1,1)	$\vec{e}_{2,2}$

Face	OuterComponent	InnerComponents
f_1	nil	$\vec{e}_{1,1}$
f_2	$\vec{e}_{4,1}$	nil

Half-edge	Origin	Twin	IncidentFace	Next	Prev
$\vec{e}_{1,1}$	v_1	$\vec{e}_{1,2}$	f_1	$\vec{e}_{4,2}$	$\vec{e}_{3,1}$
$\vec{e}_{1,2}$	v_2	$\vec{e}_{1,1}$	f_2	$\vec{e}_{3,2}$	$\vec{e}_{4,1}$
$\vec{e}_{2,1}$	v_3	$\vec{e}_{2,2}$	f_1	$\vec{e}_{2,2}$	$\vec{e}_{4,2}$
$\vec{e}_{2,2}$	v_4	$\vec{e}_{2,1}$	f_1	$\vec{e}_{3,1}$	$\vec{e}_{2,1}$
$\vec{e}_{3,1}$	v_3	$\vec{e}_{3,2}$	f_1	$\vec{e}_{1,1}$	$\vec{e}_{2,2}$
$\vec{e}_{3,2}$	v_1	$\vec{e}_{3,1}$	f_2	$\vec{e}_{4,1}$	$\vec{e}_{1,2}$
$\vec{e}_{4,1}$	v_3	$\vec{e}_{4,2}$	f_2	$\vec{e}_{1,2}$	$\vec{e}_{3,2}$
$\vec{e}_{4,2}$	v_2	$\vec{e}_{4,1}$	f_1	$\vec{e}_{2,1}$	$\vec{e}_{1,1}$

Facetten-Knoten-Inzidenzliste:

- Folge von Eckpunktindexfolgen der Dreiecke (Polygone)
- Folge von Eckpunktkoordinaten

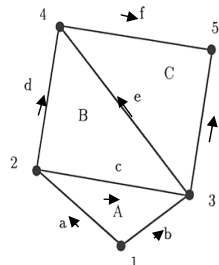
Bsp.:

Dreiecke:

1 2 3
2 3 4
3 4 5

Punkte:

1 x_1 y_1
2 x_2 y_2
3 x_3 y_3
4 x_4 y_4
5 x_5 y_5



Bemerkung: Vor allem zur Speicherung von Netzen in Dateien gebräuchlich.

Bsp.:

- STL-Format: [http://en.wikipedia.org/wiki/STL_\(file_format\)#ASCII_STL](http://en.wikipedia.org/wiki/STL_(file_format)#ASCII_STL)
- Geomview-Format: <http://www.geomview.org>

**Approximation von Kurven in Parameterdarstellung durch Polygonzüge:**

Gegeben: Eine ebene Kurve in Parameterdarstellung $\mathbf{k}(t)$, $t \in [a, b]$.

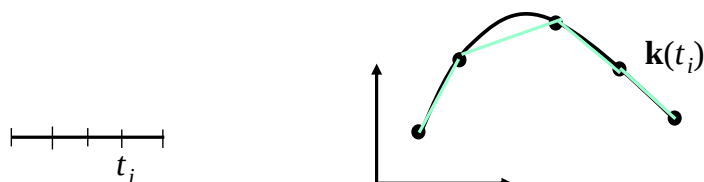
Gesucht: Eine Approximation von $\mathbf{k}(t)$ durch eine Polygonzug innerhalb einer vorgegebenen Toleranz.

Anwendung:

Zeichnen von Kurven mit Graphikbibliotheken, die nur Strecken anbieten

**Verfahren 1: Äquidistantes Unterteilen des Parameterintervalls**

Das Parameterintervall $[a, b]$ wird äquidistant in Parameterwerte t_i unterteilt. Die Kurvenpunkte $\mathbf{k}(t_i)$ mit diesen Parameterwerten werden als Eckpunkte des approximierenden Polygonzugs genommen.



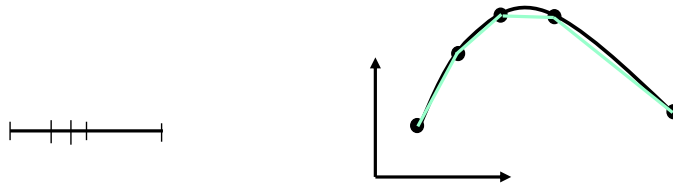
Schwierigkeit: Wahl der Schrittweite



Verfahren 2: Adaptives Unterteilen

Sukzessives Zerlegen der Kurve in zwei Teilkurven durch fortschreitendes Halbieren des Parameterintervalls $[a, b]$.

Die Unterteilung einer Teilkurve wird abgebrochen, wenn die Strecke zwischen zwei Intervallendpunkten eine gute Approximation des Kurvenstücks über dem Intervall darstellt.



Bemerkung:

Häufig hierfür verwendete Heuristik für den Abbruch der Unterteilung:

Der Abstand zwischen dem Punkt der Kurve zum Parameterwert in der Mitte des betrachteten Intervalls und dem Mittelpunkt der approximierenden Verbindungsstrecke ist kleiner als eine vorgegebene Schranke ϵ .



Approximation von Flächen in Parameterdarstellung durch Polygonnetze:

Gegeben: Eine Fläche in Parameterdarstellung $\mathbf{f}(u,v)$, $u \in [a, b]$, $v \in [c, d]$.

Gesucht: Eine Approximation von $\mathbf{f}(u,v)$ durch ein Polygonnetz innerhalb einer vorgegebenen Toleranz.

Bemerkung:

Die Approximation von Flächen in impliziter Darstellung $F(x,y,z) = 0$ wird später behandelt.

Anwendung:

Zeichnen von Flächen mit Graphikbibliotheken, die nur Polygone anbieten

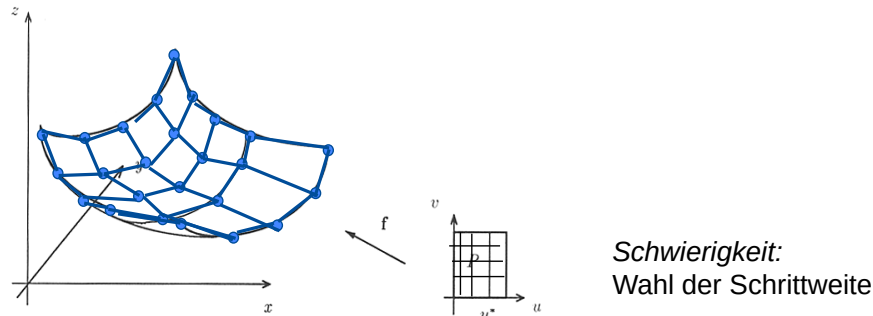


Verfahren 1: Äquidistantes Unterteilen des Parameterrechtecks

Das Parameterintervall $[a,b] \times [c,d]$ wird in Gitterpunkte (u_i, v_j) unterteilt.

Die Flächenpunkte $f(u_i, v_j)$ mit diesen Parameterwerten werden als Eckpunkte des approximierenden Vierecksnetzes genommen.

Ein Dreiecksnetz kann durch Unterteilung der Vierecksfacetten in zwei Dreiecke mittels einer Diagonalkante erreicht werden,

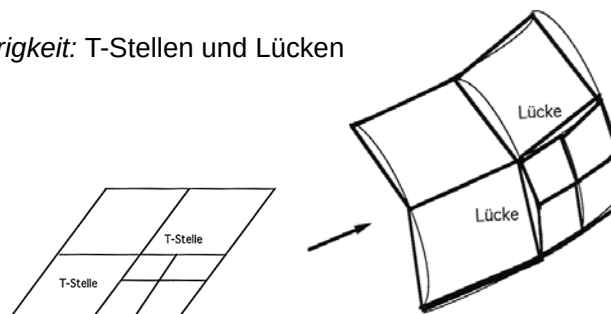


Verfahren 2: Adaptives Unterteilen

Sukzessives Zerlegen der Fläche in vier Teilflächen durch fortschreitendes Vierteln des Parameterbereichs $[a, b] \times [c, d]$.

Die Unterteilung einer Teilfläche wird abgebrochen, wenn die Teilfläche eine gute Approximation der Teilfläche über dem Parameterteilbereich darstellt.

Schwierigkeit: T-Stellen und Lücken





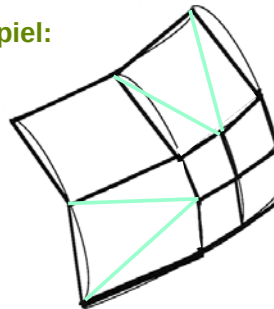
Vermeidung von T-Stellen und Lücken:

durch Unterteilen benachbarter Facetten.

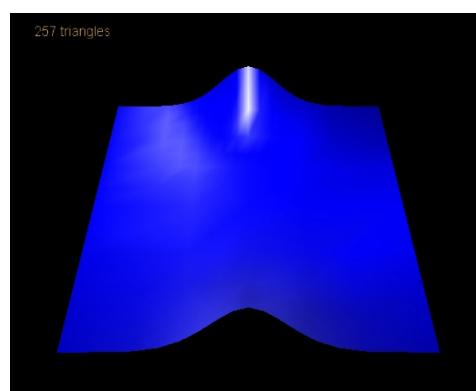
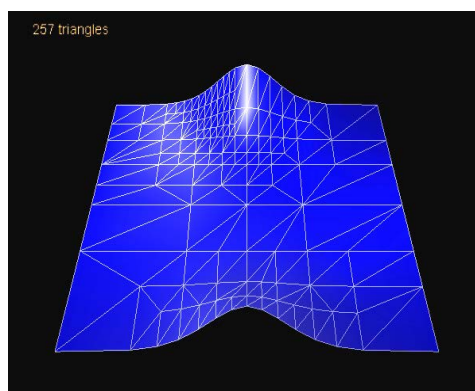
Unterteilungskonfigurationen bei maximaler Unterteilungsstufendifferenz 1 zu den Nachbarn unter Ausschluss symmetrischer Fälle:



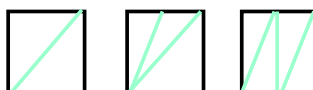
Beispiel:



Bsp.:



In diesem Bsp. verwendete
Unterteilungsmuster:



**Approximation von Kurven in impliziter Darstellung durch Polygonzüge:**

Gegeben: Eine ebene Kurve in impliziter Darstellung $F(x,y) = 0$.

Gesucht: Eine Approximation der Kurve durch eine Polygonzug innerhalb einer vorgegebenen Toleranz.

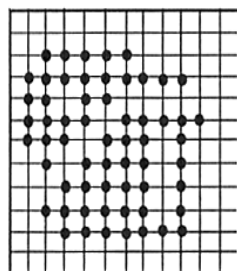
Lösungsansatz:

Verwendung des **Marching-Squares-Algorithmus**.

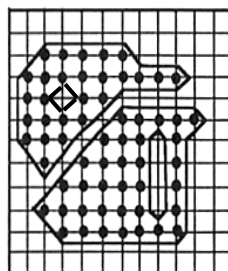
**Marching-Squares-Verfahren**

Eingabe: Ein zweidimensionales Gitter, dessen Gitterpunkte in zwei Klassen aufgeteilt sind.

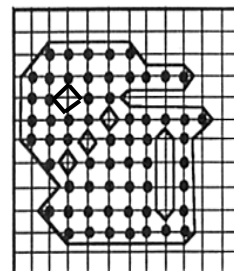
Ausgabe: Geschlossene Polygonzüge, die die Gitterpunkte der beiden Klassen trennen.

Beispiel:

Eingabe



Lösungsbsp. 1



Lösungsbsp. 2



Marching-Squares-Verfahren

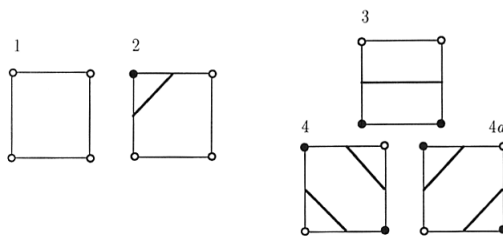
Ablauf:

BEGIN

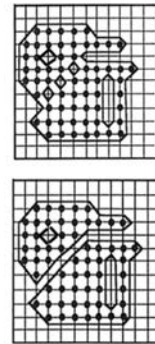
FOR alle Rasterquadrate des Gitters **DO**
finde aufgrund der Zugehörigkeit der Eckpunkte die passende
elementare Konfiguration und gib deren Strecken passend
transformiert aus

END.

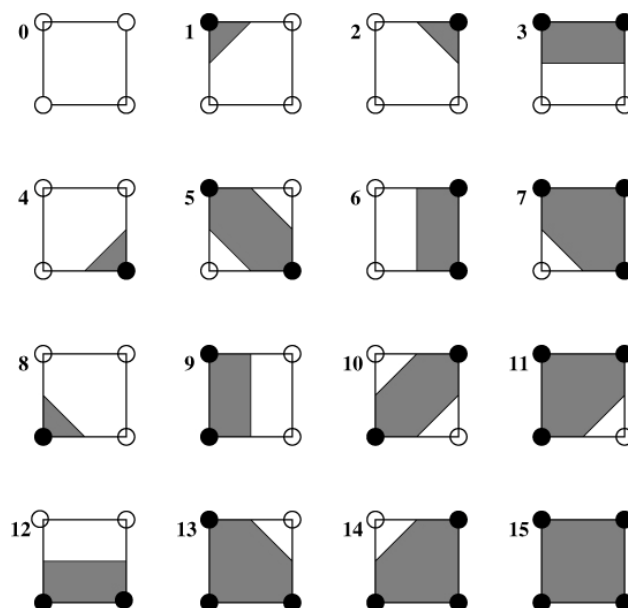
Elementare Konfigurationen (bis auf Symmetrie):



Beispiele:



Marching-Squares-Algorithmus (Konfigurationen)





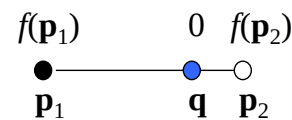
Approximation ebener impliziter Kurven $f(x,y)=0$ durch Polygonzüge:

1. Wähle ein Rechteck als Darstellungsbereich.
2. Wähle ein Rastergitter auf dem Rechteck.
3. Wende den Marching-Squares-Algorithmus an, wobei sich die zwei Eckpunktklassen als die Eckpunkte mit positivem Vorzeichen bzw. nicht positivem Vorzeichen der Funktion $f(x,y)$ an den Eckpunkten ergibt.

Lege die Kantenunterteilungspunkte $\mathbf{q}$ an die Nullstelle der linearen Interpolation der Kantenendknoten $\mathbf{p}_1, \mathbf{p}_2$ längs einer Kante:

$$\mathbf{q} = (1-\lambda) \mathbf{p}_1 + \lambda \mathbf{p}_2,$$

wobei λ die Lösung von $(1-\lambda) f(\mathbf{p}_1) + \lambda f(\mathbf{p}_2) = 0$.



Approximation von Flächen in impliziter Darstellung durch Polygonnetze:

Gegeben: Eine Fläche in impliziter Darstellung $F(x,y,z) = 0$.

Gesucht: Eine Approximation der Fläche durch ein Polygonnetz innerhalb einer vorgegebenen Toleranz.

Lösungsansatz:

Verwendung des **Marching-Cubes-Algorithmus**.



Marching-Cubes-Verfahren

Eingabe: Ein dreidimensionales Gitter, dessen Gitterpunkte in zwei Klassen aufgeteilt sind.

Ausgabe: Geschlossene Polygonnetze, die die Gitterpunkte der beiden Klassen trennen.

Ablauf:

BEGIN

FOR alle Zellen des Gitters **DO**

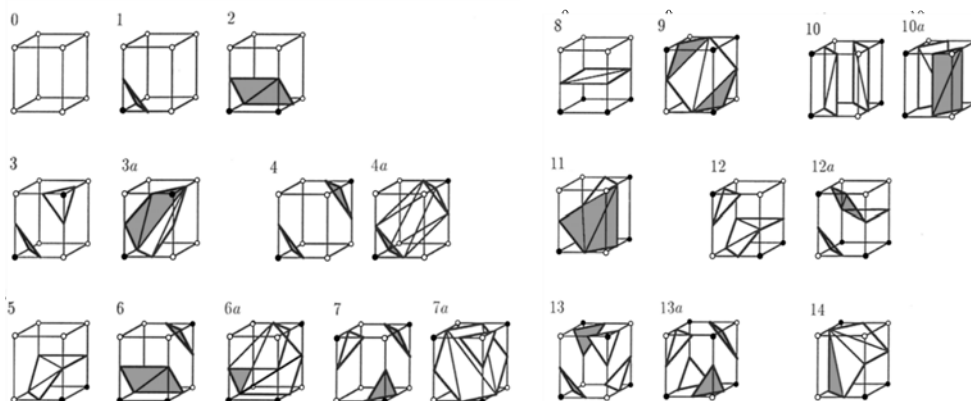
finde aufgrund der Zugehörigkeit der Eckpunkte die passende elementare Konfiguration und gib deren Dreiecke passend transformiert aus

END.



Marching-Cubes-Verfahren:

elementare Konfigurationen:





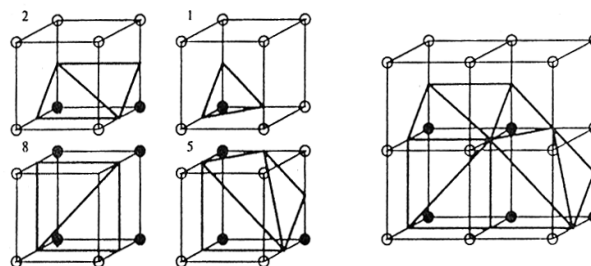
Marching-Cubes-Verfahren:



Quelle: <http://jessecolinjackson.com/portfolio/marching-cubes>



Beispiel:

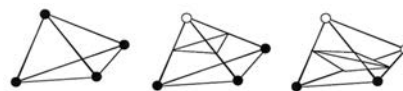


Foley, van Dam, Feiner, Hughes, Kap. 20.6

W.E. Lorensen, H.E. Cline, Marching Cubes: A High Resolution 3D Surface Construction Algorithm, Computer Graphics, 21(4) 163-169, 1987

Bemerkung:

Ein entsprechendes Verfahren kann auch für polyedrische Zellzerlegungen angewandt werden, wenn die Zellen etwa aus Tetraedern bestehen.



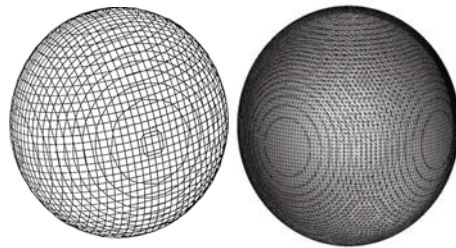


Approximation impliziter Flächen $f(x,y,z)=0$ durch Polygonnetze:

1. Wähle einen Quader als Darstellungsbereich.
2. Wähle ein Rastergitter auf dem Quader.
3. Wende den Marching-Cubes-Algorithmus an, wobei sich die zwei Eckpunktklassen als die Eckpunkte mit positivem Vorzeichen bzw. nicht positivem Vorzeichen der Funktion $f(x,y,z)=0$ an den Eckpunkten ergibt.

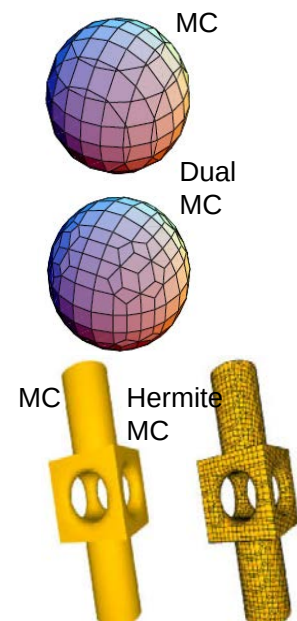
Lage der Kantenunterteilungspunkte q : wie bei impliziten Kurven ● ———— ● ○

Beispiel: Kugelapproximation für zwei unterschiedliche Gitterauflösungen



Ergänzungen, Modifikationen und Alternativen zum Marching Cubes Verfahren

- Heuristik zum Auflösen von Mehrdeutigkeiten:
G. Nielson, and B. Hamann, "The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes," Proc. of IEEE Visualization '91, pp. 29-38, 1991
- Schnelle Neuberechnung bei interaktiver Änderung von Iso-Werten:
Y. Livnat, H. Shen, and C. Johnson, "A near optimal isosurface extraction algorithm for structured and unstructured grids," IEEE Trans. on Vis. and Comp. Graph. Vol. 2, no. 1, pp. 73-84, 1996
- Duales Marching Cubes Verfahren:
S. Schaefer, J. Warren, Dual Marching Cubes: Primal Contouring of Dual Grids, Proceedings of Pacific Graphics 2004, 70-76
- Marching Cubes Verfahren unter Berücksichtigung von Normalenvektoren:
T. Ju, F. Losasso, S. Schaefer, J. Warren, Dual Contouring of Hermite Data, ACM Transactions on Graphics 21, 2002, 339-346 (SIGGRAPH 2002)

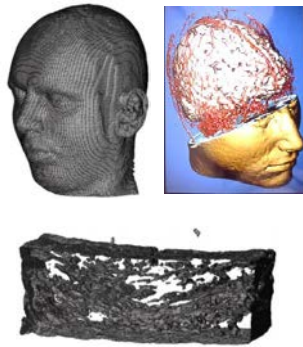


Bemerkung

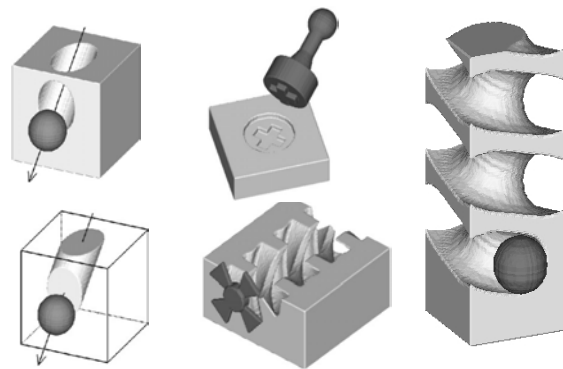
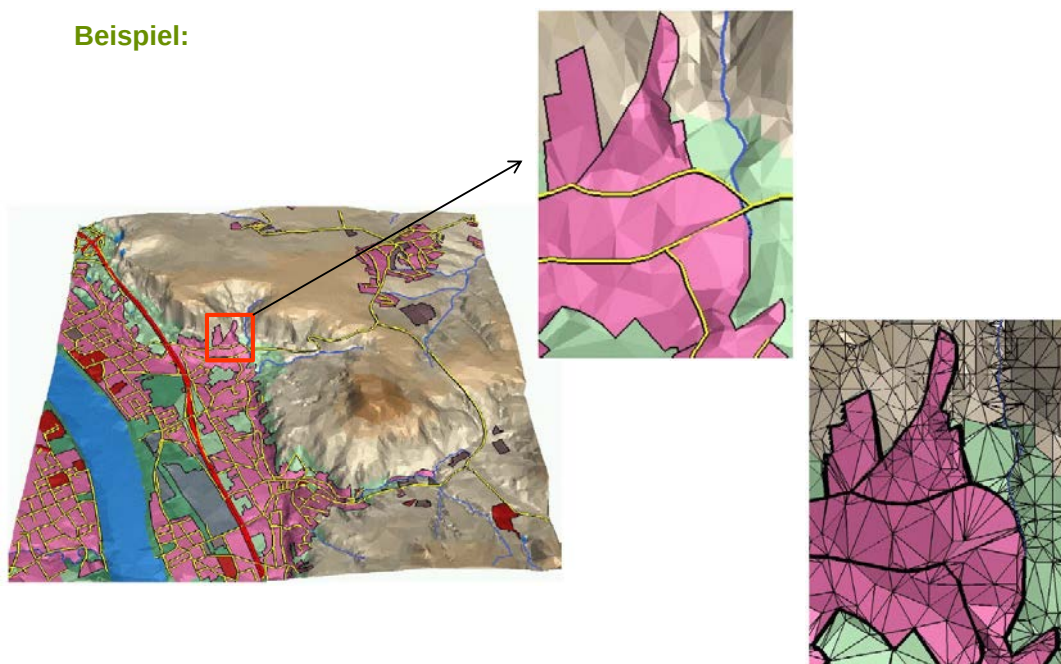
Das Marching-Cubes-Verfahren wurde ursprünglich zur Visualisierung von Rastermodellen durch Polygone entwickelt.

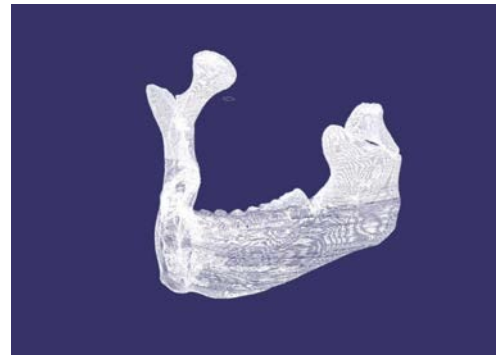
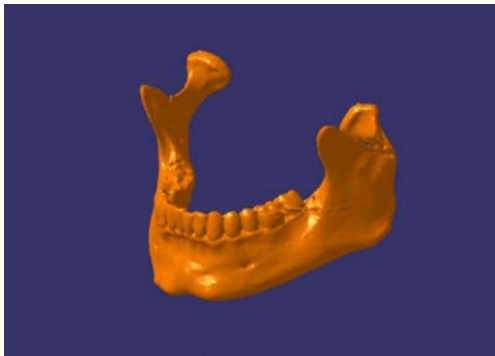
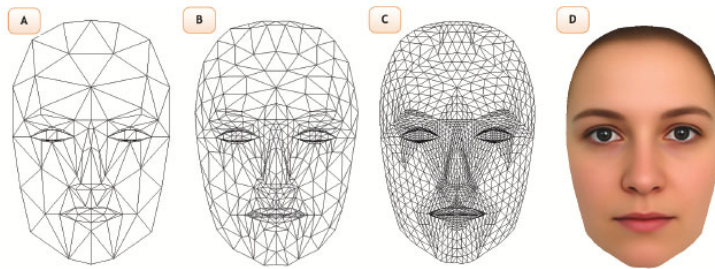
Beispiele:

Tomographie

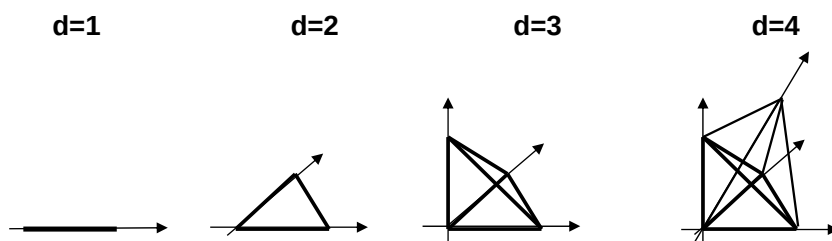


Rasterbasiertes Modellieren

**Beispiel:**

Beispiel: **d -dimensionales Simplex:**Konvexe Hülle von $d+1$ Punkten im $\mathbb{R}^d$ **Beispiel: Einheits-Simplizes**

$$S_d := \{ \mathbf{p} : \mathbf{p} = (p_1, \dots, p_d), 0 \leq p_1, \dots, p_d \leq 1, p_1 + \dots + p_d \leq 1 \}$$

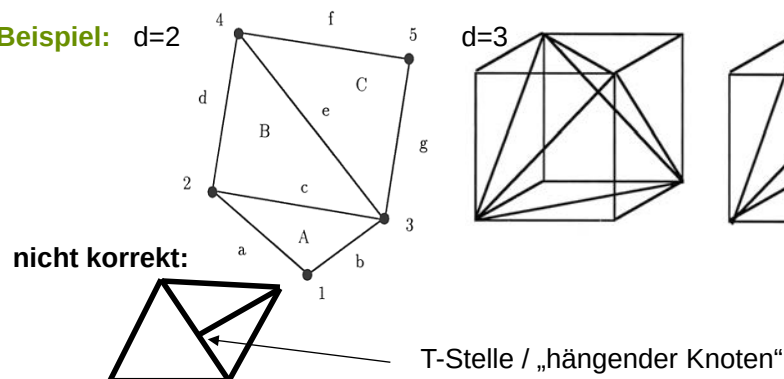


d-dimensionaler simplizialer Komplex

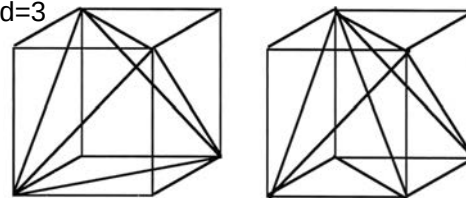
eine endliche Menge von d -dimensionalen Simplexen, sodass

- der Durchschnitt von je zwei Simplexen entweder leer oder ein Seiten-Simplex beider Simplexe ist,
- mit jedem Simplex auch alle seine Seitensimplexe zur gegebenen Simplexmenge gehören.

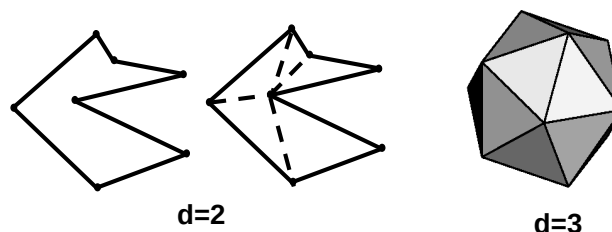
Beispiel: $d=2$



$d=3$

**Polyeder:**

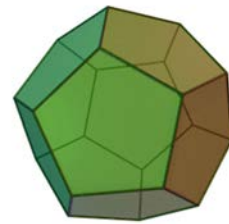
eine Punktmenge im $\mathbb{R}^d$, die durch eine d -dimensionale simpliziale Zerlegung beschrieben werden kann, dargestellt durch eine Hierarchie von Seitenpolyedern. Dabei setzt sich der Rand aus endlich vielen Seitenpolyedern zusammen, deren Rand wieder aus Seitenpolyedern und so fort, mit Eckpunkten als 0-dimensionale Polyeder.

Beispiele:

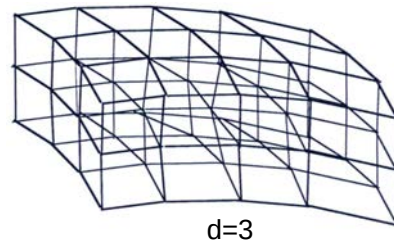
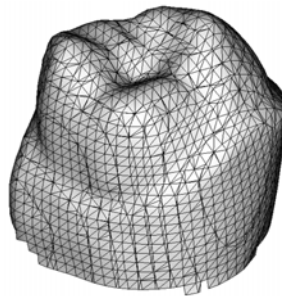
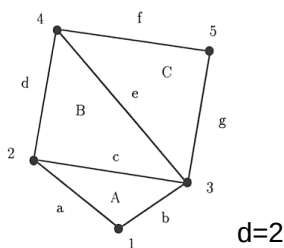
polyedrischer Komplex:

eine endliche Menge von Polyedern im mit den Eigenschaften, dass

- der Durchschnitt von je zwei Polyedern entweder leer oder ein Seitenpolyeder beider Polyeder ist,
- mit jedem Polyeder auch alle seine Seitenpolyeder zur gegebenen Polyedermenge gehören.



Dodekaeder

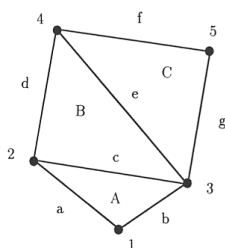
Beispiele:

d=3

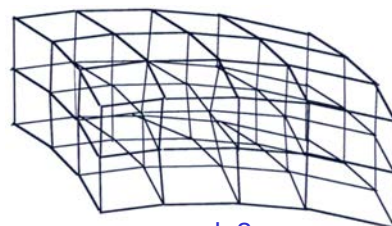
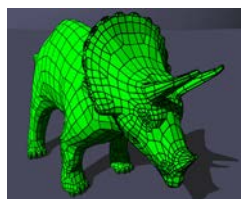
d-dimensionaler polyedrischer Komplex:

eine endliche Menge von Polyedern im $\mathbb{R}^d$ mit den Eigenschaften, dass

- der Durchschnitt von je zwei Polyedern entweder leer oder ein Seitenpolyeder beider Polyeder ist,
- mit jedem Polyeder auch alle seine Seitenpolyeder zur gegebenen Polyedermenge gehören.

Beispiele:

d=2



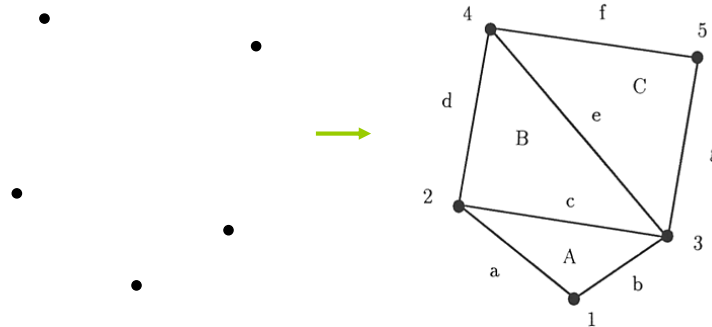
d=3

Simplex

Ein Simplex $S \in \mathbb{R}^d$ ist die konvexe Hülle von $d+1$ Punkten $\mathbf{p}_i$, die nicht alle in einer Hyperebene liegen.

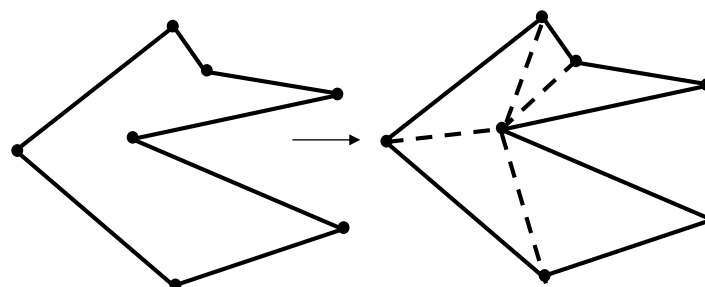
Triangulierung einer Punktmenge $\{\mathbf{p}_1, \dots, \mathbf{p}_n \in \mathbb{R}^d\}$:

simpliziale Zerlegung der konvexen Hülle von $\mathbf{p}_1, \dots, \mathbf{p}_n$, die genau die gegebenen Punkte als Eckpunkte (nulldimensionale Simplexe) hat.

Beispiel:**Polygontriangulierung**

Gegeben: Polygon P .

Gesucht: Verbindung der Knoten des Polygons durch sich nicht schneidende Sehnen, sodass eine Zerlegung des Polygoninneren in Dreiecke induziert wird.

Beispiel

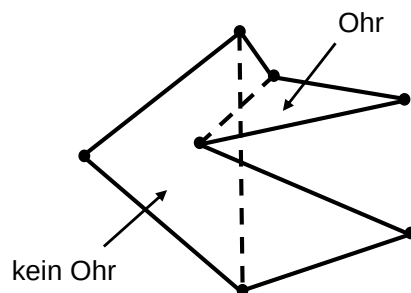
Triangulierung von Polygonen durch Ohrenabschneiden

Vorgehensweise:

sukzessives Entfernen von *Ohren* des gegebenen Polygons

Ohr: ein von einer Sehne induziertes Dreieck, das ganz im Inneren des Polygons liegt.

Behauptung (ohne Bew.): Jedes Polygon besitzt mindestens ein Ohr.



Beobachtung:

Ein von drei aufeinanderfolgenden Eckpunkten des Polygons gebildetes Dreieck ist schon ein Ohr, wenn es keinen nichtkonvexen Polygoneckpunkt enthält.

ALGORITHMUS (Polygontriangulierung)

Eingabe: ein einfaches Polygon mit Eckpunkten $p_0, \dots, p_n$.

Ausgabe: Eine Menge D von Diagonalen, die eine Triangulierung des Polygons definieren.

Datenstrukturen:

D = Menge von Diagonalen, am Anfang leer

R = Menge der nichtkonvexen Eckpunkte des Polygons

P = Eckpunkte des bisher nicht triangulierten Restpolygons, am Anfang also das ganze Polygon.

Operationen:

$PRED(p)$ = Vorgänger des Eckpunktes p auf dem Rand des Restpolygons

$SUCC(p)$ = Nachfolger des Eckpunktes p auf dem Rand des Restpolygons

$OHR(P, R, p)$ = wahr, wenn p zusammen mit $PRED(p)$ und $SUCC(p)$ ein Ohr bildet, sonst falsch.



```

Ablauf:  $p := p_2$ ;
WHILE  $p \neq p_0$  THEN
  BEGIN
    IF  $\text{OHR}(P, R, \text{PRED}(p))$  und  $P$  kein Dreieck ist
      THEN
        BEGIN
           $D := D \cup \{\text{PRED}(\text{PRED}(p), p)\}$ ;
           $P := P - \text{PRED}(p)$  (* Ohr abschneiden *)
          IF  $p \in R$  und  $p$  konvex
            THEN  $R := R - p$ ;
          IF  $\text{PRED}(p) \in R$  und  $\text{PRED}(p)$  konvex THEN
            THEN  $R := R - \text{PRED}(p)$ ;
          IF  $\text{PRED}(p) = p_0$ 
            THEN  $p := \text{SUCC}(p)$ 
          END
        ELSE  $p := \text{SUCC}(p)$  (* war kein Ohr, also weiter *)
      END.
  END.

```

**Bemerkung:**

Der Zeitaufwand des Triangulierungsalgorithmus ist $O(n \cdot k)$, k die Anzahl der nicht-konvexen Ecken. Es sind asymptotisch effizientere Algorithmen bekannt (sogar mit Zeitaufwand $O(n)$), die aber schwieriger zu implementieren sind und bei eher kleinen Polygonen höhere praktische Laufzeiten haben.

QUELLE:

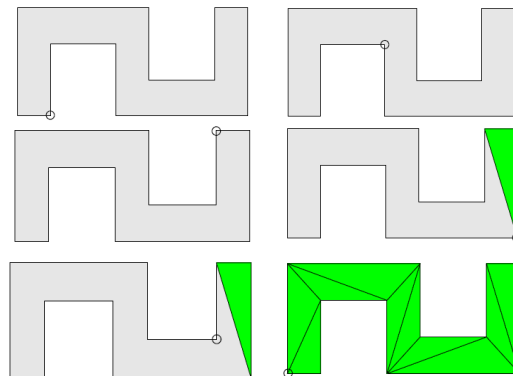
G. Toussaint, Efficient Triangulation of Simple Polygons, The Visual Computer 7 (1991) 280–295

Triangulierung von Polygonen durch Ohrenabschneiden**Trivialer Algorithmus:** $O(n^3)$

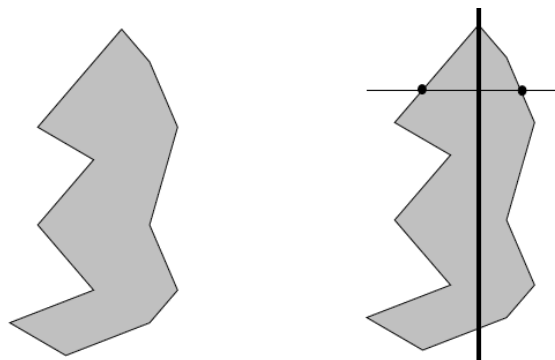
Wenn $n > 3$
 Für jede potentielle Diagonale $(i, i+2)$
 Wenn Diagonale $(i, i+2)$ ist
 Entferne Ohr bei p_i
 Wiederhole mit restlichem Polygon

Algorithmus von Kong: $O(n^2)$

Starte an Position 3
 Solange $n > 3$
 Wenn Dreieck $\text{Ohr}(i-2, i-1, i)$ ist
 Entferne Ohr
 Setze Position auf $i-2$
 Sonst gehe eine Position weiter

**Monotones Polygon**

- monoton zu einer Geraden, wenn alle Senkrechten der Geraden das Polygon max. zweimal schneiden
- zwei Seiten sind monoton
- y-monoton: monoton bezüglich der y-Achse



Monotones Polygon

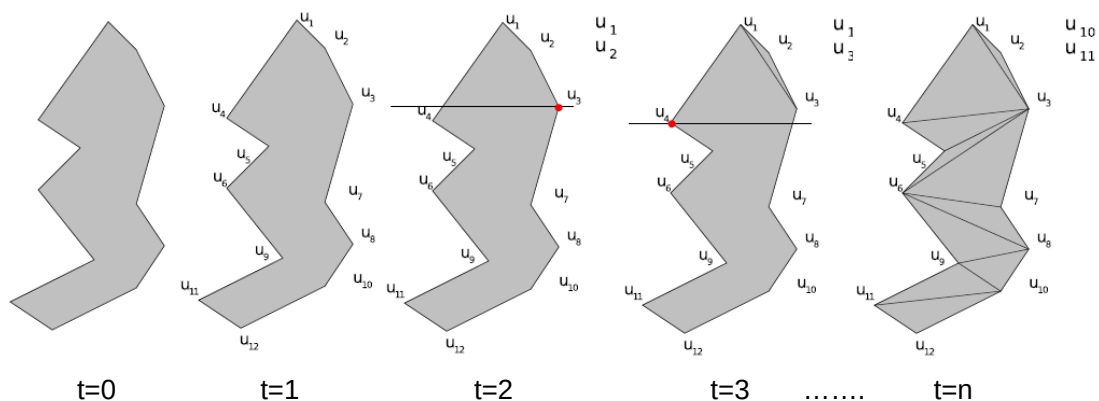
- Sweepline Algorithmus
 - in y-Richtung bei y-monotonen Polygon
 - Sortieren Punkte (Merge der Seiten) nach y (2. Kriterium x): $u_1, \dots, u_n$
- Stack S, der nicht bearbeiteten Punkte erstellen, Initialisierung mit u_1 und u_2

AlgorithmusFür $i=3$ bis n Wenn u_i auf anderer Seite als S.top Diagonale von u_i zu Punkten in S bis auf Letzten

Entferne alle Punkte aus S

 Lege u_{i-1} und u_i auf S

Sonst

 Solange S.top-1 nicht für u_i von S.top verdeckt wird Erstelle Diagonale von u_i nach S.top-1 und entferne S.top Passe letzten Punkt und u_i in SFüge Diagonalen von u_n zu Punkten in S bis auf Ersten und Letzten ein**Monotones Polygon****Beispiel**

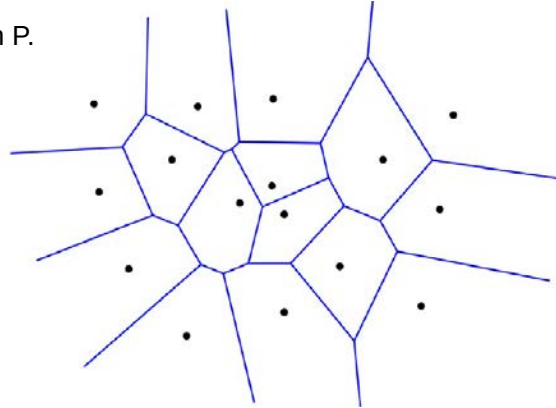
Voronoi-Diagramm

Sei $P := \{p_1, \dots, p_n\}$ eine Menge n verschiedener Punkte in der Ebene.
 Eine planare Unterteilung der Ebene mit n Facetten $V(p_i)$ mit

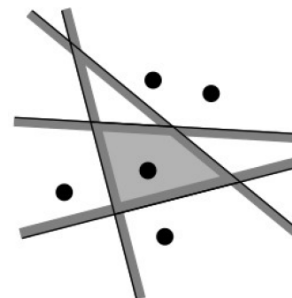
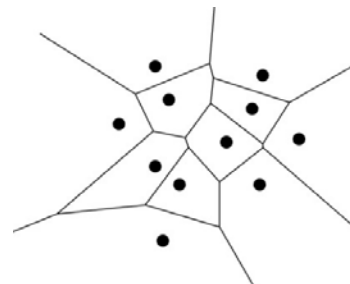
$$q \in V(p_i) \Leftrightarrow \text{dist}(q, p_i) < \text{dist}(q, p_j) \quad \forall j \neq i$$

$$\text{dist}(p, q) := \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$$

heißt Voronoi-Diagramm $\text{Vor}(P)$ von P .

**Voronoi-Diagramm - Bemerkungen**

- Es gibt Kanten mit einem sowie zwei Endpunkten.
- Es gibt offene und geschlossene Facetten bzw. Voronoizellen.
- Jede Voronoizelle entsteht bei n Punkten aus dem Schnitt von $n-1$ Halbebenen.
- Voronoizellen sind konvex und haben bis zu $n-1$ Kanten als Begrenzung.



Voronoi-Diagramm - Bemerkungen

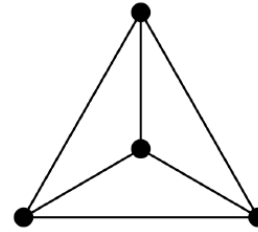
- Eulerformel für planare Graphen:

$$m_v - m_e + m_f = 2$$

m_v : Anzahl der Knoten

m_e : Anzahl der Kanten

m_f : Anzahl der Facetten



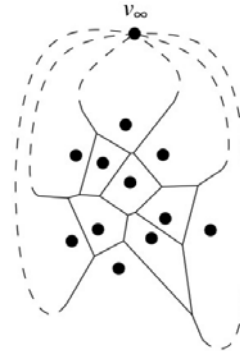
- Eulerformel für Voronoidiagramme:

m_v : Anzahl der Voronoiknoten $n_v + 1$

m_e : Anzahl der Voronoikanten n_e

m_f : Anzahl der Facetten, also n

$$(n_v + 1) - n_e + n = 2$$

**Voronoi-Diagramm - Bemerkungen**

- jede Kante hat genau zwei Knoten
- jeder Knoten hat mindestens drei Nachbarn, es folgt:

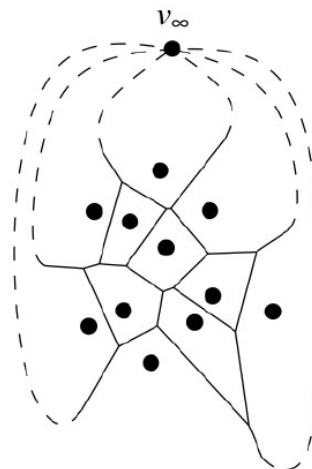
$$2n_e \geq 3(n_v + 1)$$

- Umstellen von

$$(n_v + 1) - n_e + n = 2$$

- und Einsetzen in obige Ungleichung liefert die Komplexitäten für Knoten und Kanten des Voronoi-Diagramms:

$$n_v \leq 2n - 5 \quad \text{und} \quad n_e \leq 3n - 6$$



Eulerformel für Voronoidiagramme - Beispiel

$$(n_v + 1) - n_e + n_F = 2$$

$$(4 + 1) - 9 + 6 = 2$$

$$2n_e \geq 3(n_v + 1)$$

$$2 \cdot 9 \geq 3(4 + 1)$$

$$18 \geq 15$$

$$n_v \leq 2n_F - 5$$

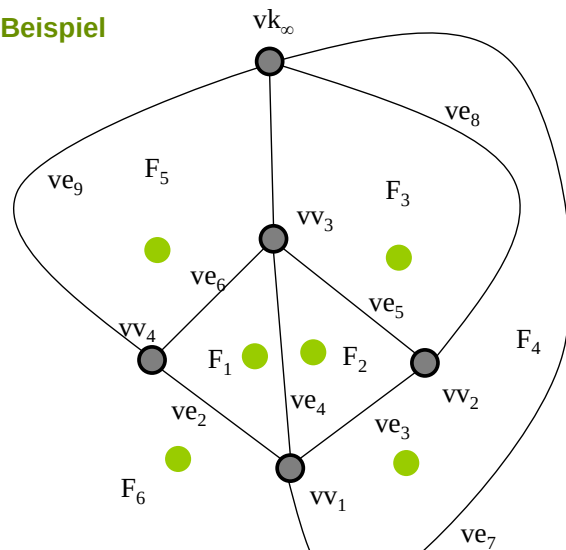
$$4 \leq 2 \cdot 6 - 5$$

$$4 \leq 7$$

$$n_e \leq 3n_F - 6$$

$$9 \leq 3 \cdot 6 - 6$$

$$9 \leq 12$$



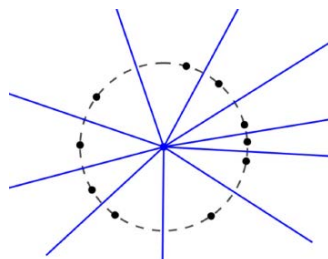
Voronoi-Knoten: vV_i

Voronoi-Kante: ve_i

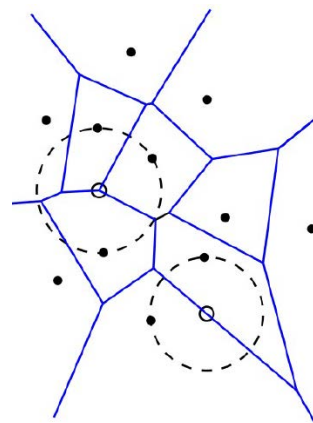
Facette: F_i

Voronoi-Diagramm - Empty Circle Property

- jeder Voronoi-Knoten ist Zentrum eines leeren Kreises durch drei Knoten der gegebenen Punktemenge
- jeder Punkt auf einer Voronoikante ist Zentrum eines leeren Kreises durch zwei Knoten der gegebenen Punktemenge



degenerierter Fall



Delaunay-Triangulierung:

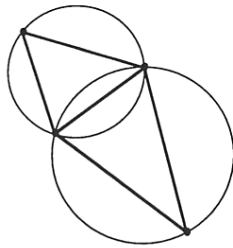
Eine Triangulierung, die nur aus Delaunay-Simplexen bezüglich der gegebenen Punkte besteht.

Delaunay-Simplex:

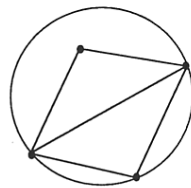
Ein Simplex mit k von n Punkten als Eckpunkte, $1 \leq k \leq \min\{n, d + 1\}$, der bezüglich der gegebenen Punkte die Umkugelbedingung erfüllt.

Beispiele:

Mögliche Triangulierungen von vier Punkten

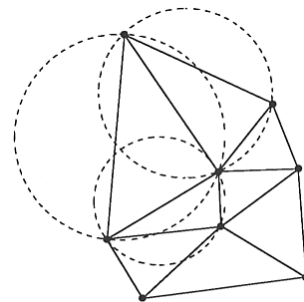


Delaunay-Triangulierung

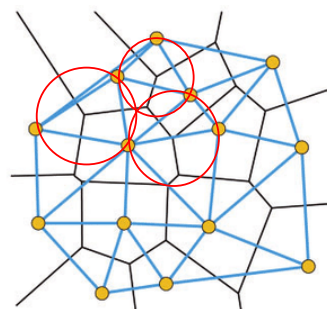
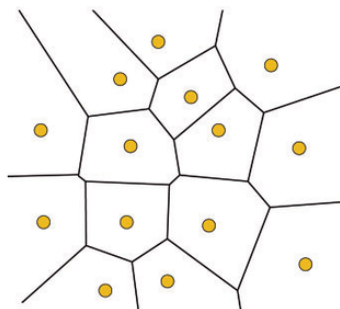


Keine
Delaunay-Triangulierung

Delaunay-Triangulierung
von mehreren Punkten

**Bemerkungen:**

1. Im Zweidimensionalen sind Delaunay-Triangulierungen diejenigen, bei denen der kleinste Winkel über alle Dreiecke maximiert wird.
2. Delaunay-Triangulierungen in beliebigen Dimensionen können durch eine Berechnung der konvexen Hülle einer um eine Dimension höheren Punktmenge berechnet werden.
3. Voronoi-Diagramm und Delaunay-Graph sind dual. Zwei Knoten sind genau dann verbunden, wenn die Voronoi-Zellen eine gemeinsame Kante haben.





Eigenschaften von Delaunay-Triangulierungen

1. Existenz:

Zu jeder endlichen Punktmenge im $\mathbb{R}^d$ gibt es eine Delaunay-Triangulierung.

2. Eindeutigkeit:

Sind keine $d + 2$ der gegebenen Punkte kosphärisch, dann ist die Delaunay-Triangulierung eindeutig.

3. Delaunay-Simplizes:

Sind keine $d + 2$ der gegebenen Punkte kosphärisch, dann

- ist eine Triangulierung genau dann eine Delaunay-Triangulierung, wenn jeder Simplex Delaunay-Simplex ist.
- sind die Simplizes der Delaunay-Triangulierung genau die Delaunay-Simplizes der gegebenen Punktmenge.

Beweis: s. nächste Folien



Transformation Delaunay-Triangulierung/konvexe Hülle

Sei $T : \mathbb{R}^d \rightarrow \mathbb{R}^{d+1}$ definiert durch

$$T(\mathbf{p}) = \begin{pmatrix} \mathbf{p} \\ \sum_{i=1}^d p_i^2 \end{pmatrix}, \quad \mathbf{p} = (p_1, \dots, p_d).$$

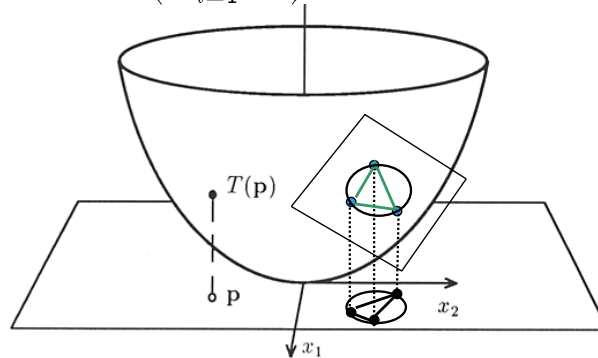
Dann ist die Delaunay-Triangulierung zu den Punkten $\mathbf{p}_1, \dots, \mathbf{p}_n \in \mathbb{R}^d$ kombinatorisch äquivalent zu der Zellzerlegung, die von den d -dimensionalen Zellen der konvexen Hülle von $T(\mathbf{p}_1), \dots, T(\mathbf{p}_n)$ induziert werden, deren äußerer Normalenvektor eine negative $(d + 1)$ -Komponente hat.

Die Delaunay-Triangulierung wird also von allen Zellen und deren Nachbarzellen der konvexen Hülle gebildet, welche von der Hyperebene aus sichtbar sind, die von den ersten d Koordinatenachsen aufgespannt wird.

Literatur:

H. Edelsbrunner, Algorithms in Combinatorial Geometry, Springer-Verlag, 1987

Beweisidee: $T(\mathbf{p}) = \left(\sum_{i=1}^d p_i^2 \right), \mathbf{p} = (p_1, \dots, p_d).$



- Netz der konvexen Hülle besteht aus Dreiecken auf dem Paraboloid.
- Dreiecke induzieren Dreiecke in der Ebene.
- Ebene durch ein Dreieck auf dem Paraboloid schneidet eine „Kuppe“ ab, die keinen der projizierten gegebenen Punkte enthält.
- Projektion des Randes der Kuppe in die Ebene ist ein Kreis.
- Kreis ist Umkreis des Dreiecks in der Ebene und enthält keinen der gegebenen Punkte.

Algorithmus (d -dimensionale Delaunay-Triangulierung)

Eingabe:

eine Punktmenge $\{\mathbf{p}_1, \dots, \mathbf{p}_n \in \mathbb{R}^d\}$.

Ausgabe:

eine d -dimensionale Delaunay-Triangulierung, dargestellt durch einen Zellinzidenzgraphen.

Ablauf:

BEGIN

berechne die konvexe Hülle von $T(\mathbf{p}_1), \dots, T(\mathbf{p}_n)$;

bestimme den Teilgraphen des Zellinzidenzgraphen, der die von der zur $(d+1)$ -Achse senkrechten Koordinatenebene aus sichtbaren Zellen beschreibt

END.

**Bemerkungen:**

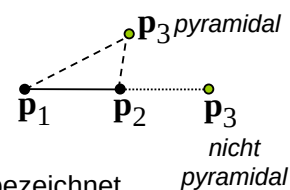
- *Lexikographisches Sortieren* bedeutet, dass zunächst nach der ersten Koordinate, bei gleicher erster Koordinate nach der zweiten Koordinate und so fort sortiert wird.
- Bei der Hinzunahme des nächsten Punktes $\mathbf{p}_i$ sind zwei Fälle zu unterscheiden: $\mathbf{p}_i$ kann in dem von den Punkten $\mathbf{p}_1, \dots, \mathbf{p}_{i-1}$ aufgespannten linearen Unterraum, deren *affiner Hülle* $\text{aff}\{\mathbf{p}_1, \dots, \mathbf{p}_{i-1}\}$, liegen oder nicht.

Affine Hülle einer Menge: der kleinste affine Unterraum (Punkt, Gerade, Ebene, ...), der die gegebene Menge enthält.

Beispiel:

Der dritte hinzugenommene Punkt $\mathbf{p}_3$ kann auf der Geraden durch $\mathbf{p}_1$ und $\mathbf{p}_2$ liegen oder nicht.

Die zweite Situation wird als *pyramidale Hinzunahme* bezeichnet.

**Algorithmus** (d -dimensionale konvexe Hülle)

Eingabe: n Punkte

Ausgabe: die konvexe Hülle von $\mathbf{q}_1, \dots, \mathbf{q}_n$

Ablauf:

BEGIN

bestimme die lexikographisch sortierte Anordnung $\mathbf{p}_1, \dots, \mathbf{p}_n$ der gegebenen Punkte $\mathbf{q}_i$;

initialisiere die konvexe Hülle als leere Menge;

FOR $i := 1$ **TO** n **DO**

{ füge $\mathbf{p}_i$ zur konvexen Hülle von $\mathbf{p}_1, \dots, \mathbf{p}_{i-1}$ hinzu }

END.



Algorithmus (Beneath-Beyond-Algorithmus, pyramidale Hinzunahme)

Eingabe: die konvexe Hülle zu $p_1, \dots, p_{i-1}$, $p_i \notin \text{aff}\{p_1, \dots, p_{i-1}\}$.

Ausgabe: die konvexe Hülle von $p_1, \dots, p_i$.

Ablauf:

BEGIN

konstruiere die Punktmenge, deren Rand eine Pyramide ist, deren Grundfläche die bisher konstruierte konvexe Hülle ist und deren Spitze der neu hinzukommende Punkt ist;

gib die Punktmenge als Ergebnis aus;

END.



Algorithmus (Beneath-Beyond-Algorithmus, nichtpyramidale Hinzunahme)

Eingabe: die konvexe Hülle zu $p_1, \dots, p_{i-1}$,
ein weiterer Punkt $p_i \notin \text{aff}\{p_1, \dots, p_{i-1}\}$.

Ausgabe: die konvexe Hülle von $p_1, \dots, p_i$.

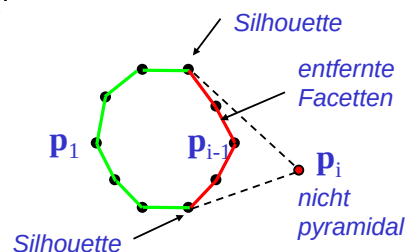
Ablauf:

BEGIN

entferne alle Facetten der bisher konstruierten konvexen Hülle, die von p_i aus sichtbar sind;

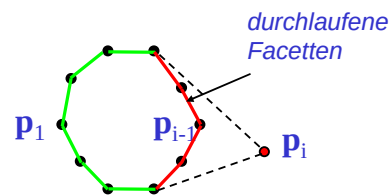
verbinde p_i durch Facetten mit dem Rand der resultierenden Fläche, d.h. den Silhouettenfacetten der bisher konstruierten konvexen Hülle

END.

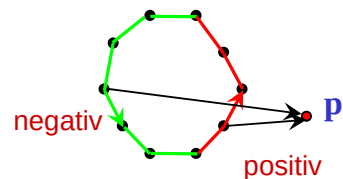


Bemerkungen:

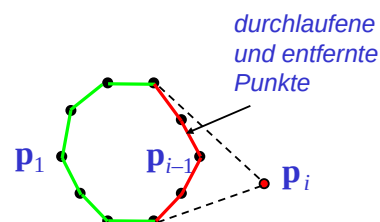
- Die von $\mathbf{p}_i$ aus sichtbaren Flächen und die entsprechenden Silhouettenfacetten lassen sich durch Ablaufen der Oberfläche des Randes der konvexen Hülle ausgehend von $\mathbf{p}_{i-1}$ finden, da $\mathbf{p}_{i-1}$ von $\mathbf{p}_i$ aus sichtbar ist.



- Wenn die Normalenvektoren der volldimensionalen Facetten alle nach außen zeigen, lässt sich die Entscheidung durch das Vorzeichen eines Skalarprodukts treffen.

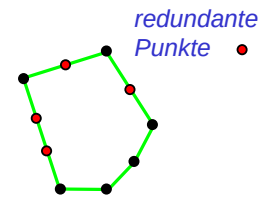
**Bemerkungen:**

- Für eine zweidimensionale Punktmenge gibt es eine Realisierung des Algorithmus mit $O(n \log n)$ Rechenzeit.
- Die Sortierphase hat einen Zeitaufwand von $O(n \log n)$.
- Die gesamte Hinzunahmephase hat einen Zeitaufwand von $O(n)$. Der Grund ist, dass die Rechenzeit proportional zur Anzahl der Punkte ist, die insgesamt beim Anfügen des nächsten Punktes inspiziert werden. Wenn die Inspektion ausgehend von $\mathbf{p}_{i-1}$ erfolgt, wird im Wesentlichen jeder inspizierte Punkt auch entfernt. Da nicht mehr als n Punkte existieren, ergibt sich der Zeitaufwand $O(n)$.



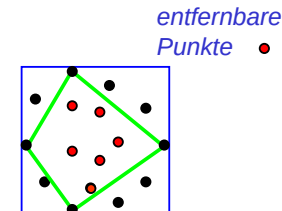
Bemerkungen:

- Der Algorithmus kann so realisiert werden, dass redundante Facetten, d.h. Facetten, die zu größeren Facetten zusammengefasst werden können, vermieden werden.



- Manchmal ist es lohnend, Vortests anzuwenden, mit denen unter Umständen Punkte im Inneren der konvexen Hülle entfernt werden können,

Bsp.: Punkte im Inneren der konvexen Hülle der Punkte der gegebenen Punktmenge, die auf dem achsenparallelen Hüllquader liegen.

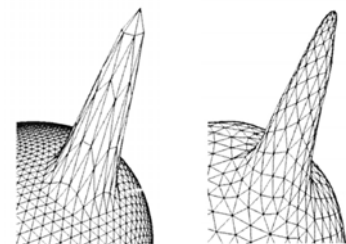
**Literatur:**

- H. Edelsbrunner, Algorithms in Combinatorial Geometry, Springer-Verlag, 1987
- S. Abramowski, H. Müller, Geometrisches Modellieren, B.I.Wissenschaftsverlag, 1991, Kap. 8.3

Geometrische Netzqualität:

Form der Dreiecke:
z.B. Maximierung des Flächeninhalts relativ zur Kantenlänge

Beispiel (Computer Graphics Forum 19(3), S. 253):

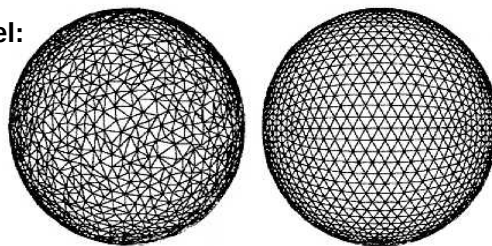


unausgeglichene Dreiecke ausgeglichene Dreiecke

Kombinatorische Netzqualität:

möglichst regulärer Knotengrad (bei Dreiecksnetzen z.B. = 6).

Beispiel:



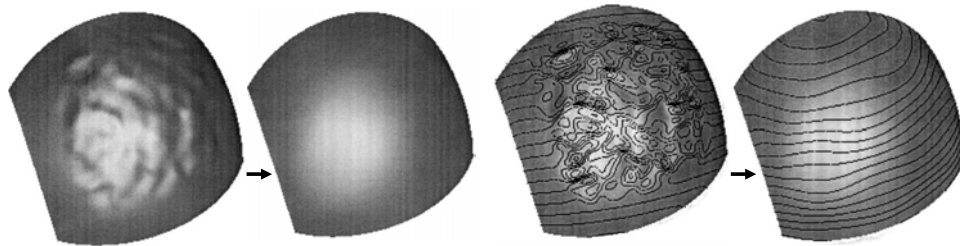
irreguläres Netz semi-reguläres Netz (fast überall Grad 6)

LITERATUR:

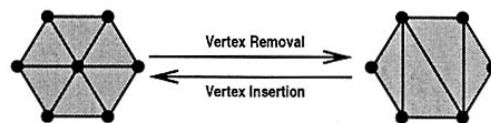
L. Kobbelt, S. Bischoff, M. Botsch, K. Kähler, Ch. Rössl, R. Schneider, J. Vorsatz, Geometric Modeling Based on Polygonal Meshes, Research Report, Max-Planck-Institut für Informatik, July 2000

Flächenqualität:

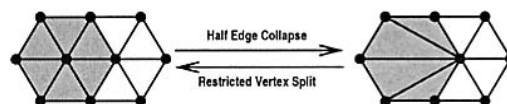
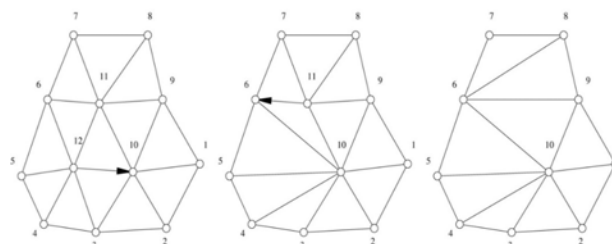
- Glattheit (smoothness):
Definition der Glattheit über Differenzierbarkeit
- Ausgeglichenheit (fairness):
Definition der Ausgeglichenheit über krümmungsbasierte Funktionale

**Operatoren zur Netzveränderung:**

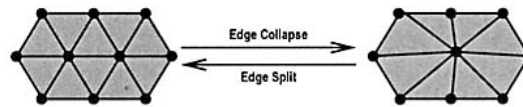
- Knotenentfernung (vertex removal):



- Halbkantenkontraktion (half-edge contraction/collapse):

**Bsp.:**

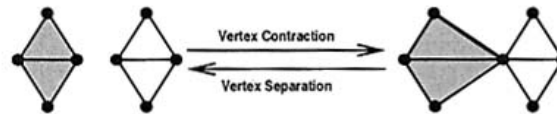
- Kantenkontraktion (edge contraction/collapse):



Bemerkung:

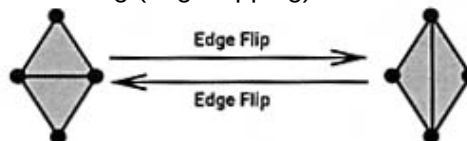
Der neue Punkt kann etwa als Mittel der zwei Endpunkte der zusammengezogenen Kante bestimmt werden.

- Knotenkontraktion (vertex contraction/collapse):



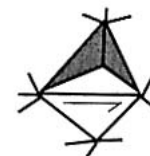
Bemerkung: Die Knotenkontraktion kann die Netztopologie verändern

- Kantenvertauschung (edge flipping):



Bemerkungen:

1. Zu den Operationen gibt es die jeweils inversen Operationen. Sie sind in den Abbildungen aufgeführt.
2. Bei der Anwendung der Operationen muss darauf geachtet werden, dass zu garantierende Eigenschaften des Netzes nicht verloren gehen. So verändert beispielsweise die Knotenkontraktion die Netztopologie.



Durch Anwendung einer Halbkantenkontraktion in der gezeigten Weise ergeben sich Dreiecke, die in einer Ebene aufeinander liegen.

**Umvernetzungsalgorithmus von Kobbelt et al.:****QUELLE:**

L. Kobbelt, T. Bareuther, H.-P. Seidel,
Multiresolution Shape Deformations for Meshes with Dynamic Vertex
Connectivity, Computer Graphics Forum 19(3), 2000, C-249-C-259

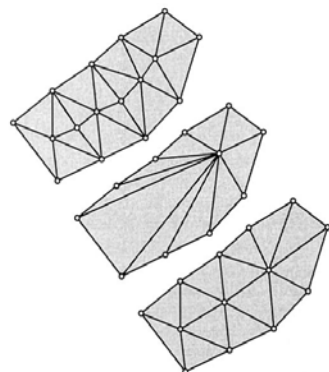
Ziel:

Umvernetzung, sodass möglichst

- keine Kante kürzer als eine gegebene Schranke $\varepsilon_{\min} > 0$
- keine Kante länger als eine gegebene Schranke $\varepsilon_{\max} > 2\varepsilon_{\min}$
- der Knotengrad gleich 6 ist
- lange und dünne Dreiecke vermieden werden.

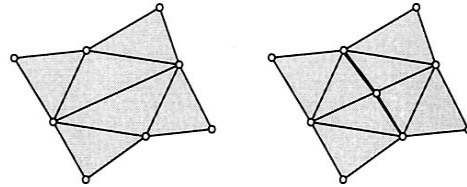
**Algorithmus (Kobbelt et al.)****Ablauf:**

1. Entferne alle Kanten, die kürzer als $\varepsilon_{\min} > 0$ sind, durch Halbkantenkontraktion. Dabei wird der Endknoten mit geringerem Knotengrad in den mit höherem Grad geschoben.



Das Entstehen von Knoten hohen Grades wie im mittleren Bild wird vermieden. Das untere Bild zeigt das Ergebnis, das sich aus dem oberen Bild ergibt.

2. Unterteile alle Kanten, die länger als $\varepsilon_{\max} > 0$ sind, durch Einfügen eines Knotens und Halbierung der inzidenten Dreiecke.



Bem.: Hier wird die Forderung $\varepsilon_{\max} > 2\varepsilon_{\min}$ benötigt, damit nicht wieder zu kurze Kanten entstehen

3. Für alle adjazenten Dreiecke $\Delta(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3)$, $\Delta(\mathbf{p}_3, \mathbf{p}_2, \mathbf{p}_4)$, bei denen sich der totale Knotengrad

$$\sum_{i=1}^4 (d(\mathbf{p}_i) - 6)^2$$

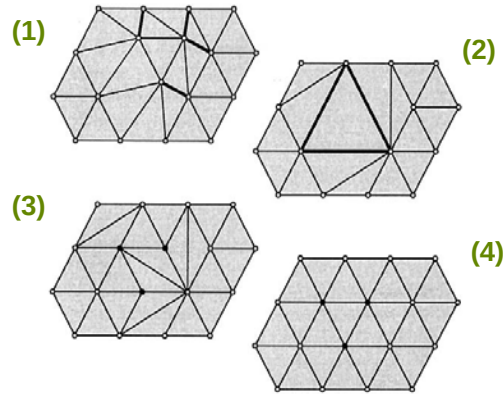
wobei $d(\mathbf{p}_i)$ der Knotengrad von $\mathbf{p}_i$, bei Vertauschen der gemeinsamen Kante $(\mathbf{p}_2, \mathbf{p}_3)$ verringert, führe einen Kantentausch aus.

Bem.: Dadurch soll die Anzahl der Knoten mit Grad 6 erhöht werden.

4. Falls Kanten entstanden sind, deren Länge nicht im Intervall $(\varepsilon_{\min}, \varepsilon_{\max})$ liegen, dann wende das Verfahren auf das aktuelle Netz erneut an.

Beispiel:

- (1) Eingabernetz
- (2) Ergebnis nach Entfernen der zu kurzen Kanten
- (3) Ergebnis nach Entfernen der zu langen Kanten
- (4) Ergebnis nach dem Kantentausch zur Gradreduktion

**Algorithmus: Netzausgleich durch diskreten Informationsfluss (Taubin)****Ziel:**

Elimination von feinen Details, insbesondere von feinen Unebenheiten („Rauschen“), die bei der Netzgenerierung durch Abtasten oder Interpolations-/Approximationsverfahren entstanden sind.

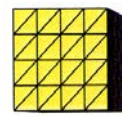
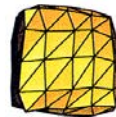
Beispiel (aus Vollmer et al., Computer Graphics Forum 18(3)):



(künstlich)
verrauschter
Würfel



Ergebnisse des Netzausgleich



Original zum
Vergleich

LITERATUR: L. Kobbelt, S. Bischoff, M. Botsch, K. Kähler, Ch. Rössl, R. Schneider, J. Vorsatz, Geometric Modeling Based on Polygonal Meshes, Research Report, Max-Planck-Institut für Informatik, July 2000

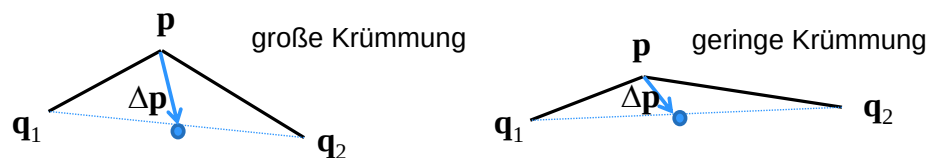
Vorgehensweise: Verschiebung der Netzknoten in Richtung des Vektors, der durch eine diskrete Version des Laplace-Operators berechnet wird.

Diskreter Laplace-Operator:

$$\Delta \mathbf{p} := \left(\sum_{\mathbf{q} \in N(\mathbf{p})} \lambda(\mathbf{p}, \mathbf{q}) \cdot \mathbf{q} \right) - \mathbf{p}$$

mit $\lambda(\mathbf{p}, \mathbf{q}) := 1 / |N(\mathbf{p})|$ wird auch als uniformer diskreter Beltrami-Laplace-Operator auf Netzen bezeichnet. Er charakterisiert das Krümmungsverhalten von Netzen.

Zweidimensionale Illustration für Polygonzüge:



Algorithmus Diskreter Informationsfluss (Taubin)

1. Verschiebe die Knoten $\mathbf{p}$ des gegebenen Netzes nach $\mathbf{p}'$, wobei

$$\mathbf{p}' := \mathbf{p} + t \cdot \Delta \mathbf{p}$$

mit

$$\Delta \mathbf{p} := \left(\sum_{\mathbf{q} \in N(\mathbf{p})} \lambda(\mathbf{p}, \mathbf{q}) \cdot \mathbf{q} \right) - \mathbf{p}$$

$N(\mathbf{p})$ ist die Menge der zu $\mathbf{p}$ adjazenten Knoten.

$\lambda(\mathbf{p}, \mathbf{q})$ sind Gewichte mit

$$\sum_{\mathbf{q} \in N(\mathbf{p})} \lambda(\mathbf{p}, \mathbf{q}) = 1$$

Der Faktor $0 < t \leq 1$ wird benötigt, um die Stabilität des Verfahrens zu steuern.

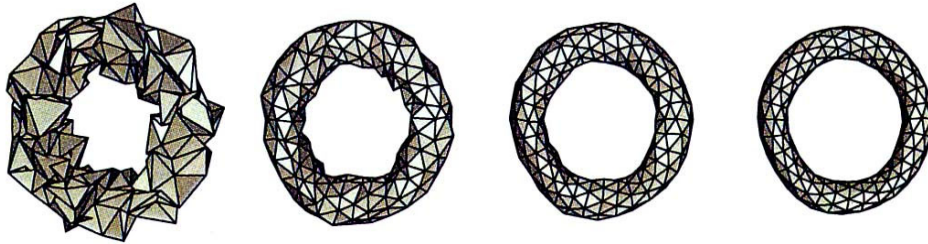
2. Iteriere das Verfahren, bis die Fläche hinreichend glatt ist.

Laplace'sche Glättung

Verfahren des diskreten Informationsflusses mit

$$t = 1 \text{ und } \lambda(\mathbf{p}, \mathbf{q}) := 1 / |N(\mathbf{p})|.$$

Beispiel: Ein künstlich verrauschter Torus (links) und drei Iterationen der der Laplace'schen Glättung. Nachteil: der Torus schrumpft.

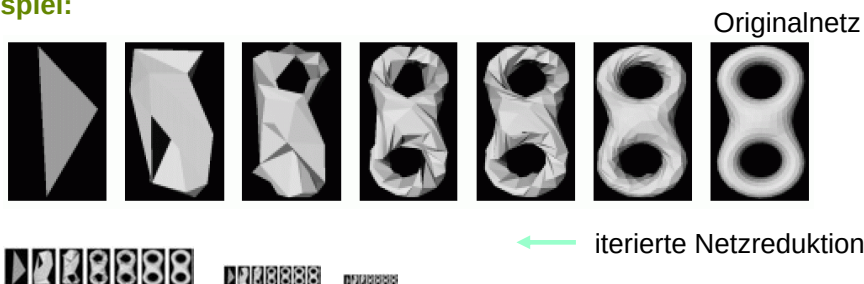
**Bemerkung:**

Durch passende Wahl von t kann das Schrumpfen bei gleichzeitiger Glättung gemindert werden.

Netzreduktion:

Eingabe: ein Netz M , eine Fehlerschranke $\varepsilon > 0$ bezüglich einer Fehlermetrik.

Ausgabe: ein Netz M' , das weniger Knoten als M hat und dessen Abweichung von M kleiner als ε ist.

Beispiel:

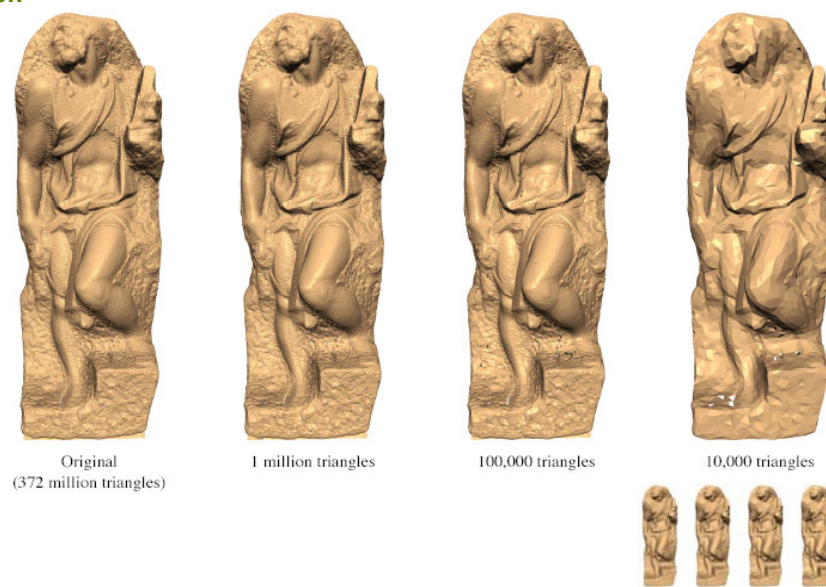
**Anwendungen:**

- Anpassung an Rechnerressourcen
z.B. Speicher, Kommunikationsbandbreite, interaktive Bilderzeugung
- progressive Datenübertragung
- hierarchische Darstellung, mehrfachaufgelöste (multi-resolution, level-of-detail)-Darstellung

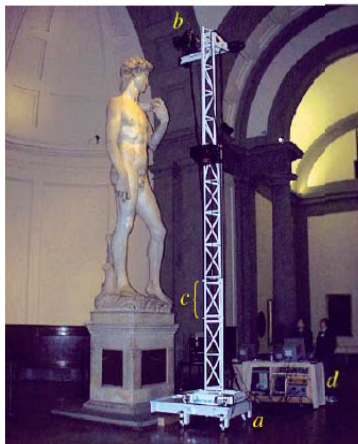
LITERATUR:

C. Gotsman, S. Gumhold, L. Kobbelt, Simplification and Compression of 3D Meshes, in: *Tutorials on Multiresolution in Geometric Modeling*, A. Iske, E. Quak, M.S. Floater (eds.), Springer-Verlag, Berlin, 2002

L. Kobbelt, S. Campagna, H.-P. Seidel, A general framework for mesh decimation, *Proc. Graphics Interface*, 1998, 105-114

**Beispiel:**

Beispiel: The Digital Michelangelo Project, <http://graphics.stanford.edu/projects/mich>



Datenerfassung durch Laser-Scanning

Beispiel: Sequentielle Übertragung



Beispiel: Progressive Übertragung



**Algorithmus (Inkrementelle Netzdezimierung)**

Eingabe: ein Netz M , eine Fehlerschranke $\varepsilon > 0$ bezüglich einer Fehlermetrik.

Ausgabe: ein Netz M' , das sich durch Dezimierung aus M ergibt und dessen Abweichung von M kleiner als ε ist.

Vorgehensweise: Anwendung einer Folge von ausgewählten Operationen zur Netzveränderung, die das Netz lokal verändern, unter Beachtung des Approximationsfehlers

Datenstrukturen:

eine Priority Queue Q für die anzuwendenden Operationen, sortiert nach aufsteigenden Kosten für die Operationen

**Ablauf:**

Für alle möglichen Operationen op :

$c :=$ Fehler, der durch Anwendung von op entsteht;

wenn $c < \varepsilon$ dann

füge (op, c) in Q ein;

Wiederhole:

wende die Operation op_{opt} in Q mit dem geringsten Fehler an;

aktualisiere Q ;

bis Q leer ist;

gib das Ergebnisnetz als M' aus.

Bemerkung:

Von der Aktualisierung von Q sind alle Operationen betroffen, deren Einflussbereich sich durch die Anwendung von op_{opt} verändert.
Ferner wird op_{opt} aus Q entfernt.

Fehlermaß:*Aspekte:*

- gemessene Eigenschaften: Geometrie (Distanz, Ausgeglichenheit),
- lokales bzw. globales Maß: Fehlerauswertung von Schritt zu Schritt oder Vergleich mit dem Originalnetz